AI Digitalise

Deep Dive: every folder, every file, every line

Digitalise Agency

2026-08-11

Contents

1 What this project is **2**
1.1 The problem it was built to solve . . . 2
1.2 The split between this repository and the live site . . . 2
1.3 Reading order, and why it is not alphabetical . . . 3
1.4 A note on three characters . . . 3

2 The ten-minute mental model **4**
2.1 The one paragraph that matters . . . 4

3 Background a beginner needs **5**
3.1 What happens when someone opens the page . . . 5
3.2 Tags, attributes, classes and selectors . . . 6
3.3 Custom properties: colours defined once . . . 6

4 Repository layout **7**

5 Architecture **7**

6 Data flow, end to end **8**

7 The file-by-file walkthrough **9**
7.1 `new-design-reference/index.html` . . . 9
7.2 `new-design-reference/css/site.css` . . . 30
7.3 `new-design-reference/js/main.js` . . . 55
7.4 `new-design-reference/netlify.toml` . . . 63
7.5 `new-design-reference/assets/ASSETS.md` . . . 64
7.6 `.github/workflows-disabled/deploy.yml.disabled` . . . 67

8 Generated and third-party files **73**

9 Configuration and secrets **73**
9.1 Secrets . . . 73

10 Dependencies **74**

11 How to run it **74**
11.1 What you should see . . . 75

12 The agent-facing surface **75**

13 Deployment **76**
13.1 Why it is disabled, and what would happen if it were not . . . 76

14 Honest findings **76**
14.1 Findings about correctness . . . 77
14.2 Findings about dead or contradictory code . . . 77
14.3 Findings about unfinished work . . . 77
14.4 Findings about accessibility . . . 77

15 The project's story **78**
15.1 Deliverables . . . 78
15.2 Timeline and effort . . . 78
15.3 Challenges . . . 79
15.4 Outcomes . . . 79
15.5 What to do (judgement) . . . 79
15.6 What not to do (judgement) . . . 80
15.7 If you were starting over (judgement) . . . 80
15.8 Open threads . . . 80

16 Glossary **81**

A Appendix A: the file ledger **82**

1 What this project is

This repository holds a marketing web page for **ai.digitalise.agency**: a single-page site whose job is to persuade a visitor that Digitalise Agency can build AI systems for them, and then get that visitor to book a call.

It is six files and 1,591 lines. There is no framework, no build step, no package manager, no database and no server-side code. A web browser can open the page straight off a disk. That makes it one of the simplest projects in the workspace, and a good one to read first if you have never read code before.

The one thing to understand before anything else

This repository does *not* contain the web page that is currently being served at ai.digitalise.agency. The design here was built, then deliberately parked, and the machinery that used to publish it was switched off. Something else is serving that address today. Section 1.2 shows the evidence, and the whole document is written with that split in mind.

1.1 The problem it was built to solve

An agency that sells AI services needs a page that a prospective client can be sent to. The page has to say what is on offer, show that other people have been happy, and make booking a conversation a single click. That is the entire purpose. Everything in these six files serves one of those three jobs.

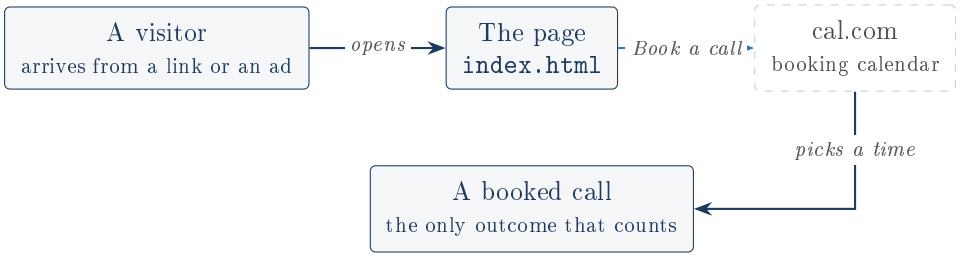


Figure 1: The whole project in one picture. The dashed box is a third-party service the agency does not run. Everything in this repository exists to move a visitor from the left box to the bottom box.

1.2 The split between this repository and the live site

The last commit made to this repository, `da508a7` on 2026-07-29, did two things in one move: it pushed every file down into a folder named `new-design-reference/`, and it renamed the automatic publishing workflow so that it could no longer run. The person who made that commit explained why in the commit message itself, and it is worth quoting because it is the single most important fact about this repository:

> *"The live site is the generated agency variant built from the portfolio repo, not this one. Parking this design for reference and turning the workflow off so it cannot overwrite the real site."*

That claim was checked against the live address on 2026-08-11 while this document was being written. The address answers, and it does not serve this page:

What was checked	What came back
HTTP status of `https://ai.digitalise.agency/`	`200`, the site is up
Page title served	*Digitalise Agency* followed by *AI & Full-Stack Engineering*
Page title in this repository	*Digitalise AI: Done-for-you AI systems for entrepreneurs*
Canonical URL declared by the live page	`https://salmanadnan.com/`

Table 1: Verified by fetching the live address on 2026-08-11. The titles do not match, so the live page is not the page in this repository, exactly as commit `da508a7` said.

The live site tells search engines to index a different domain

The page currently served at ai.digitalise.agency declares `<link rel="canonical" href="https://salmanadnan.com/">`. A canonical link is an instruction to a search engine saying "the real home of this content is over there, index that instead of this". So the live ai.digitalise.agency is telling Google to credit salmanadnan.com for its content.
That is almost certainly a copy-and-paste accident in whatever repository produces the live page, not a decision. It is out of scope for *this* repository, which cannot fix it, but anybody reading this document is likely to be the person who can. Verified by fetching the page on 2026-08-11.

Jargon: Repository, often shortened to "repo"

A folder of files whose complete history is recorded, so that every change ever made can be listed, inspected, and undone. The recording is done by a program called *git*. When this document says "this repository", it means the folder `AI Digitalise` and the history stored alongside it.

1.3 Reading order, and why it is not alphabetical

Files are explained in the order that makes each one understandable when you reach it, not in the order a file listing would print them:

1. `new-design-reference/index.html` first, because it is the page itself. Every other file exists to serve it.
2. `new-design-reference/css/site.css` second, because the page names it and is unreadable without it.
3. `new-design-reference/js/main.js` third, because it animates what the first two files have already put on screen.
4. `new-design-reference/netlify.toml` fourth: a hosting setting for a host that was never used.
5. `new-design-reference/assets/ASSETS.md` fifth: a checklist of work that was never finished.
6. `.github/workflows-disabled/deploy.yml.disabled` last, because it is about publishing, which only makes sense once you know what is being published.

1.4 A note on three characters

Three characters used in this repository's source files cannot be printed by the typeface this document uses. Rather than silently drop them, they are printed as the closest stand-in, listed here so that no quoted line misleads you:

In the real file	Printed here as	Where it appears
A long horizontal rule (U+2500)	a hyphen	comment banners in the CSS and JavaScript
A double horizontal rule (U+2550)	an equals sign	comment banners in the HTML
A heavy black heart (U+2764)	a heart symbol	one decorative tile in the HTML

Table 2: Character substitutions used in every quoted listing in this document.

The middle dot (U+00B7), which appears in the page footer and in several file header comments, prints exactly as it does in the file and is not substituted.

2 The ten-minute mental model

If you read only one page of this document, read this one.

A web page is made of three kinds of file, and this project has exactly one of each. That is not a coincidence: it is the oldest and simplest way to build a web page, and the project chose it deliberately.

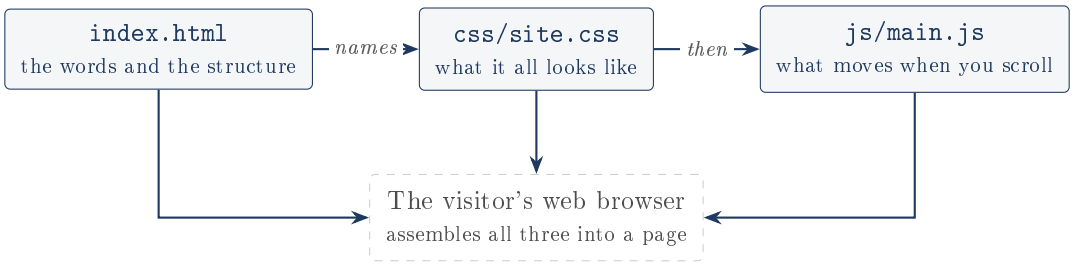


Figure 2: The three-file model. The browser downloads all three and combines them. Nothing runs on a server: everything happens on the visitor's own machine.

Jargon: HTML

Short for HyperText Markup Language. It is the file that holds the actual content of a page (the headings, the paragraphs, the links) wrapped in labels called *tags* that say what each piece of content *is*. A tag looks like `<h1>` to open and `</h1>` to close. HTML says what things are; it says almost nothing about what they look like.

Jargon: CSS

Short for Cascading Style Sheets. It is the file that decides appearance: colour, size, spacing, position, and movement. It works by naming a kind of element ("every heading", "anything labelled `btn`") and listing the appearance rules that apply to it. The word "cascading" means that when two rules disagree, there is a defined pecking order that decides which one wins.

Jargon: JavaScript

A programming language that runs inside the browser, after the page has loaded. It is what makes a page *react*: to a scroll, a click, a keypress. Unlike HTML and CSS, which describe things, JavaScript gives instructions that happen in sequence.

2.1 The one paragraph that matters

`index.html` lays out seven stacked bands down the page: a navigation bar, a full-screen hero with floating tiles behind it, a row of four big numbers, a five-step pipeline diagram, a grid of six past projects, three client result cards, a grid of six services, a booking panel, and a footer. `site.css`

gives all of that its dark navy look, its rounded corners and its layout, using a small set of named colours defined once at the top. `main.js` adds three behaviours and nothing else: the navigation bar grows a dark glassy background once you scroll past 40 pixels, the four big numbers count up from zero when they scroll into view, and most blocks fade upward into place as they appear. The "Book a call" buttons are ordinary links: two of them jump down the page to the booking panel, and the button inside that panel opens a cal.com calendar in a new tab.

Why "no build step" is a real design decision

Most modern web projects are written in one form and then *compiled* into another before they can be published, by a tool the developer has to install and maintain. This project skips that entirely: the files you read are byte for byte the files a browser receives. The benefit is that anybody can open, understand and change it with nothing but a text editor. The cost is that there is nothing to catch a mistake before a visitor sees it, and no way to share code with another page except by copying it.

3 Background a beginner needs

This section defines everything the project assumes you already know. If you know it, skip to section 4.

3.1 What happens when someone opens the page

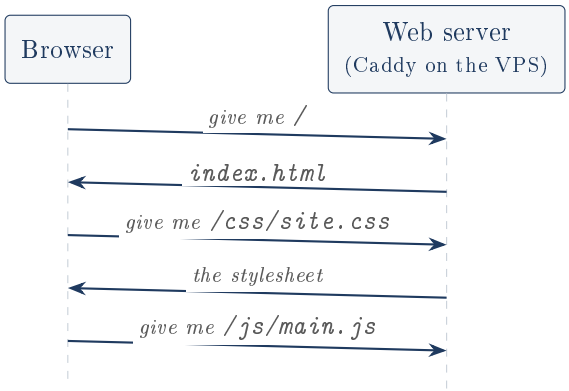


Figure 3: The request sequence. The browser reads `index.html` first, discovers the other two files named inside it, and asks for those next. This is why the `<link>` and `<script>` tags matter so much: they are the only reason the browser knows the other files exist.

Jargon: Web server

A program that sits on a computer that is always on, waits for requests for files, and sends those files back. This project is served by a web server called *Caddy* running on the agency's rented computer. Caddy does no thinking here: it hands over files exactly as they sit on disk.

Jargon: VPS

Virtual Private Server: a computer the agency rents from a hosting company and controls remotely, rather than one sitting in an office. The deploy workflow in section 12 names its address as `46.202.194.85`.

3.2 Tags, attributes, classes and selectors

Four words do most of the work in this project.

A **tag** marks a piece of content: `<p>` is a paragraph, `<section>` is a major band of a page, `<a>` is a link. Most tags come in pairs that open and close, with content in between.

An **attribute** is extra information written inside the opening tag, as a name and a value: in `<a href="#book">`, the attribute `href` has the value `#book`, meaning "this link points at the thing named book on this same page".

A **class** is a label you invent and attach to any number of elements, with the `class` attribute. It carries no meaning to the browser; it exists so the CSS can find those elements again.

A **selector** is how CSS names what it wants to style. `.btn` means "every element labelled `btn`". `.btn-primary:hover` means "every element labelled `btn-primary`, but only while the mouse is over it".

Jargon: Element

One thing on the page, created by a tag: a heading, a button, a box. When this document says "element", it means one of these, not a chemical one.

3.3 Custom properties: colours defined once

At the top of the stylesheet is a block that gives names to values: `-accent: #1967ff;` means "from now on, the name `-accent` stands for this particular blue". Everywhere else, the stylesheet writes `var(-accent)` instead of the raw colour. Change the definition once and every use changes with it.

Jargon: Hex colour

A colour written as a hash sign and six characters, like `#1967ff`. The six characters are three pairs giving the amount of red, green and blue, each from `00` (none) to `ff` (maximum). `#1967ff` is a strong blue: little red, a moderate amount of green, almost full blue.

Jargon: Custom property, also called a CSS variable

A name defined in CSS whose value can be reused anywhere. It always starts with two hyphens when defined, and is read back with `var(...)`. This project defines twenty of them and reads most of them dozens of times.

4 Repository layout

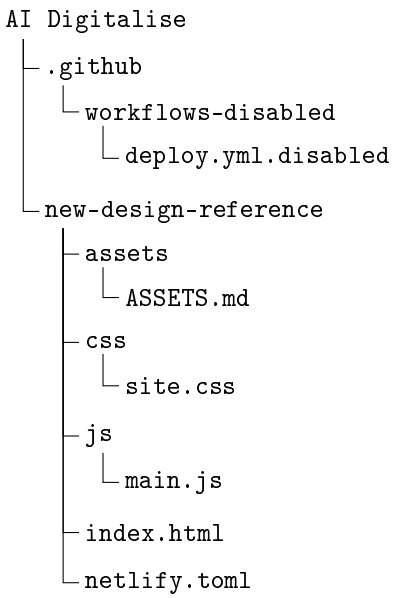


Figure 4: The complete repository. Nothing is folded away: these six files are everything git tracks. The complete list with line counts is in Appendix A.

Two things about this shape are unusual and both are deliberate.

First, **every single file lives under** `new-design-reference/`. On a normal web project, `index.html` sits at the top so that a web server pointed at the folder finds it immediately. Here it is one level down, which means a server pointed at this folder would find no home page at all. That is the parking described in section 1.2: the folder name is a message to the next person, saying "this is a reference, not a live site".

Second, **the automation folder is misspelled on purpose**. GitHub only runs files it finds in a folder called exactly `.github/workflows`. This one is called `.github/workflows-disabled`, and the file inside it ends in `.disabled` rather than `.yml`. Either change alone would be enough to stop it running. Both were made, which tells you the intent was not to pause it briefly.

Jargon: Hidden folder

A folder whose name begins with a dot, like `.github`. On Mac and Linux these are hidden from ordinary file listings by default. Nothing else is special about them; the dot is just a convention meaning "tooling lives here, not content".

5 Architecture

There is no architecture in the usual sense, because there is no running program to have one. What exists instead is a set of relationships between six files, and those are worth drawing once before walking them.

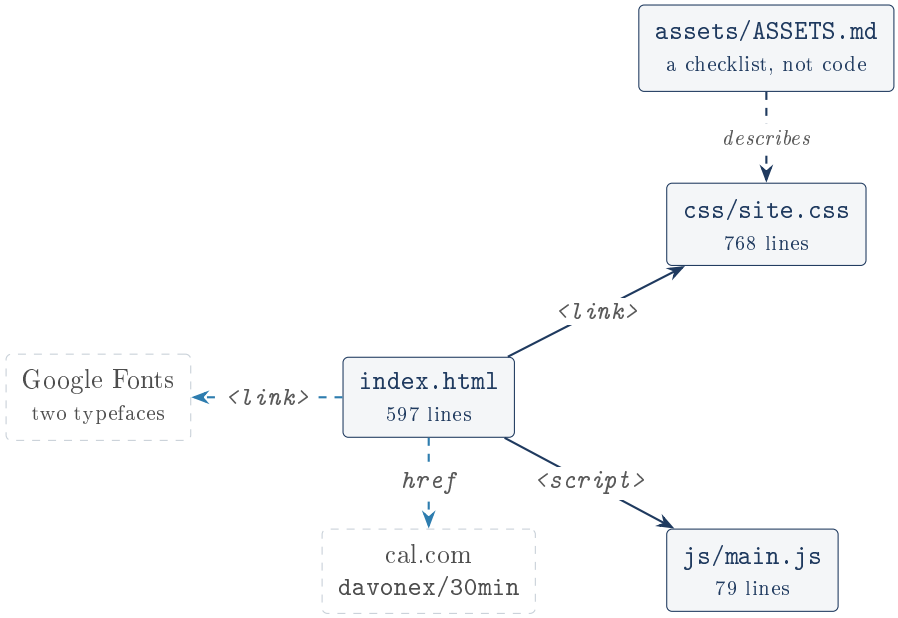


Figure 5: What names what. Solid arrows are files this project owns; dashed boxes are services on somebody else's computers. `netlify.toml` and the deploy workflow are not shown because nothing on the page refers to them: they are instructions for machines that publish the page, not part of it.

The important structural point is that **the arrows only go one way**. `index.html` knows the names of the other two files. Neither of them knows anything about it: `site.css` styles class names that it hopes exist, and `main.js` looks for elements that may or may not be there. Every one of its lookups is written to do nothing quietly if the element is missing. That is why the page does not break when a section is deleted, and also why a typo in a class name fails silently rather than loudly.

Silent failure is the price of this simplicity

Nothing anywhere checks that the class names in `index.html` and the selectors in `site.css` agree. Rename `.hero-badge` in one file and not the other and the page still loads, still scores as "working", and simply looks wrong. Section 14 lists three places where this has already happened.

6 Data flow, end to end

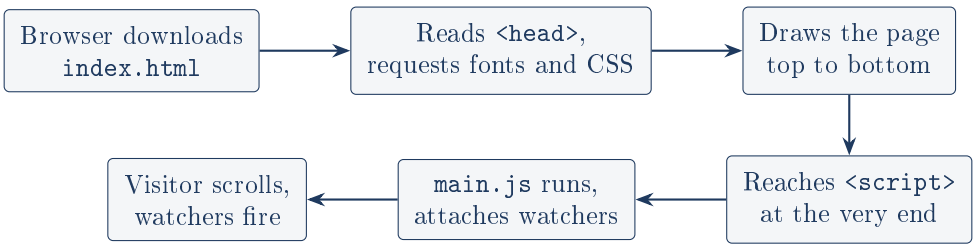


Figure 6: The full life of one page view. Note that the JavaScript is the last thing to run, because its `<script>` tag is the last thing in the document. That ordering is deliberate and is explained in section 7.3.

There is no data flow beyond this. No form is submitted, nothing is saved, nothing is sent anywhere. The only outbound step a visitor can trigger is following the cal.com link, at which point they have left this page entirely.

Nothing measures whether the page works

There is no analytics of any kind: no counter, no tag manager, no event when somebody presses "Book a call". So the question this page exists to answer, "does it get bookings", cannot be answered from the page itself. Any evidence would have to come from the cal.com side. Whether that is a privacy choice or an oversight is not recorded anywhere in the repository, so this document does not guess.

7 The file-by-file walkthrough

Every file git tracks gets its own subsection below, quoted from its first line to its last. Nothing is summarized away and nothing is skipped.

7.1 `new-design-reference/index.html`

HTML, 597 lines. The page itself: every word a visitor reads, and the structure that holds those words. If you delete every other file in this repository, this one still opens in a browser and still says everything the project wants to say. It would simply look plain.

Where this came from: new-design-reference/index.html

Hand-written. It arrived complete in the very first commit, `0d77d51` on 2026-07-28 ("Launch ai.digitalise.agency"), at the path `index.html`, and was moved to its present path in `da508a7` on 2026-07-29 with no change to its contents. There is no generator banner, no framework scaffold, and no `package.json` that could have produced it.
Its icons are a different matter and are worth naming precisely. Every icon in the file is an inline SVG drawing whose path data matches the Heroicons outline set, a widely used free icon library: the shapes, the `24 24` view box, the `stroke-linecap` and `stroke-linejoin` attributes and the `1.5`-based coordinates are that library's house conventions. **The file carries no attribution comment saying so**, so this is stated as an inference from the path data, not as a documented fact. **Confidence: high for hand-written, inferred for the icon origin.**

Why it exists

Without this file there is no page. The stylesheet styles class names that would belong to nothing, and the JavaScript would search for elements that do not exist and quietly do nothing, which is exactly how it is written to behave.

Line by line

Lines 1 to 17: the head, which the visitor never sees

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4  <meta charset="UTF-8" />
5  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6  <title>Digitalise AI: Done-for-you AI systems for entrepreneurs</title>
7  <meta name="description" content="Done-for-you AI systems that answer, follow up, and publish,
       so your business keeps growing without adding headcount." />
8  <meta property="og:title" content="Digitalise AI" />
9  <meta property="og:description" content="Done-for-you AI systems that answer, follow up, and
       publish." />
10 <meta property="og:type" content="website" />
11 <meta name="theme-color" content="#05070d" />
12
```

```
13 <link rel="preconnect" href="https://fonts.googleapis.com" />
14 <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
15 <link href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@500;600;700&family=
       Inter:wght@400;500;600;700&display=swap" rel="stylesheet" />
16 <link rel="stylesheet" href="/css/site.css" />
17 </head>
```

Listing 1: `index.html`, lines 1–17

Line 1 is not a tag at all. `<!DOCTYPE html>` is a declaration that tells the browser "interpret this as modern HTML". It is a historical artefact: without it browsers fall back to a compatibility mode that imitates the bugs of software from the 1990s. It must be the first thing in the file, and it takes no closing partner.

Line 2 opens the document and declares its language as English with `lang="en"`. That attribute is small and does real work: screen readers use it to choose a pronunciation, and search engines use it to decide who to show the page to.

Line 3 opens the `<head>`. Everything between here and line 17 is information *about* the page rather than content *of* it. Nothing in this block appears on screen.

Line 4 declares the character encoding as UTF-8, which is the scheme that lets a file contain any character in any language. This matters concretely here: the page contains a middle dot and a heart symbol, and without this line those would render as mojibake. It should appear as early as possible, because the browser has to guess the encoding until it is told.

Jargon: Meta tag

A tag inside `<head>` carrying a fact about the page rather than content for the page. It has no closing partner and displays nothing. Different meta tags are read by completely different audiences: browsers, search engines, and social networks each look for their own.

Line 5 is the viewport declaration and it is the single line that makes the page work on a phone. `width=device-width` tells the browser to treat the page width as the real width of the device, and `initial-scale=1.0` says do not zoom out to start with. Without it, a phone pretends to be about 980 pixels wide and shrinks everything, which is why old sites look like tiny unreadable newspapers on a phone. Every media query in the stylesheet depends on this line being present.

Line 6 is the `<title>`: the text in the browser tab and the blue headline in a search result. Note it for later, because it is how section 1.2 proved this page is not the live one.

Line 7 is the description, which search engines often show as the grey text under the title. It is 118 characters, comfortably inside the roughly 155 that get displayed.

Lines 8 to 10 are Open Graph tags, a convention invented by Facebook and now read by nearly every chat app and social network to build the preview card when somebody pastes a link. Note that `og:title` here says only "Digitalise AI", shorter than the real title on line 6, which is a sensible choice: preview cards truncate.

The Open Graph set is incomplete in the way that matters most

The three tags present are `og:title`, `og:description` and `og:type`. The two missing ones are `og:image` and `og:url`. Without `og:image`, a pasted link produces a preview card with no picture, which is the difference between a link that gets clicked and one that does not. `ASSETS.md` lists `og-image.png` as required before launch, and it was never created. See section 7.5.

Line 11 sets `theme-color` to `#05070d`, the same near-black used as the page background. Mobile browsers paint their address bar this colour, so the browser chrome blends into the page instead of framing it in white.

Line 12 is blank. Blank lines have no effect on how HTML renders; they are purely for the human reading the file, and here the author uses them to separate the page's identity from its

external resources.

Lines 13 and 14 are `preconnect` hints. They tell the browser: you will shortly need to talk to these two Google servers, so start the handshake now rather than waiting. Establishing a secure connection takes several network round trips, and doing that early rather than at the moment of need typically saves a few hundred milliseconds. The `crossorigin` attribute on line 14 is required on the second one, because font files are fetched under stricter rules than stylesheets, and a preconnect made without it would open a connection the font request cannot reuse. That subtlety catches out experienced developers constantly.

Line 15 actually fetches the fonts: two typefaces, Space Grotesk in three weights and Inter in four. `display=swap` at the end is the important part. It tells the browser to render text immediately in a fallback typeface and swap in the real one when it arrives, rather than hiding the text for up to three seconds while it waits. The visible cost is that text reflows slightly; the benefit is that a visitor on a slow connection can read the page at all.

Jargon: Font weight

How heavy a typeface looks, on a numeric scale where 400 is normal and 700 is bold. Requesting `wght@500;600;700` downloads three separate files. Each weight requested is another download, which is why the list is short.

Line 16 links the stylesheet. This is the path discussed in section 11: the leading slash in `/css/site.css` means "from the top of the site", which is correct when served by a web server and wrong when the file is opened directly off a disk.

Line 17 closes the head.

Lines 18 to 35: the navigation bar

```
18 <body>
19
20 <!-- ================================================
21      NAV
22 ================================================ -->
23 <nav class="nav" aria-label="Site navigation">
24   <div class="nav-inner">
25     <a href="/" class="nav-logo" aria-label="Digitalise AI home">
26       <svg viewBox="0 0 28 28" fill="none" aria-hidden="true">
27         <rect width="28" height="28" rx="8" fill="#1967ff"/>
28         <path d="M7 14c0-3.866 3.134-7 7-7s7 3.134 7 7-3.134 7-7 7" stroke="#fff" stroke-width
       ="2.2" stroke-linecap="round"/>
29         <circle cx="14" cy="14" r="2.5" fill="#fff"/>
30       </svg>
31       digitalise <span class="logo-ai">AI</span>
32     </a>
33     <a href="#book" class="btn btn-primary">Book a call</a>
34   </div>
35 </nav>
```

Listing 2: `index.html`, lines 18–35

Line 18 opens `<body>`: everything from here to line 596 is what a visitor actually sees.

Line 19 is blank, and lines 20 to 22 are a comment banner. Anything between `<!-` and `-->` is ignored entirely by the browser. This file uses seven of these banners as signposts, one per major band of the page, each drawn with a rule of repeated double-line characters. They are the author's table of contents, and they are the reason a 597-line file stays navigable.

Line 23 opens the navigation bar. Two things are happening. `<nav>` is a *semantic* tag: it does not look different from a plain `<div>`, but it announces to screen readers and search engines that this is the site's navigation. `aria-label="Site navigation"` gives that region a spoken name, so a screen-reader user hearing a list of page regions hears something meaningful.

Jargon: ARIA

Accessible Rich Internet Applications: a set of attributes, all starting with `aria-`, that describe a page to assistive software such as screen readers. They change nothing visually. `aria-label` supplies a name; `aria-hidden="true"` tells assistive software to skip something entirely because it is purely decorative.

Line 24 opens a `<div>` labelled `nav-inner`. A `<div>` is a generic container with no meaning at all; it exists purely to be styled. The reason for a second container inside `<nav>` is a common layout pattern: the outer element spans the full window width so its background can stretch edge to edge, while the inner one is capped at a maximum width and centred, so the logo and button line up with the content further down the page.

Line 25 opens a link to `/`, the site's home page. Since this *is* the home page it reloads the page, which is the conventional behaviour of a logo. Its `aria-label` matters here: the link's visible content is a drawing plus the words "digitalise AI", and without the label a screen reader would announce the drawing awkwardly.

Lines 26 to 30 are the logo, drawn in SVG.

Jargon: SVG

Scalable Vector Graphics: a picture described as shapes and coordinates rather than as a grid of coloured dots. Because it is instructions rather than pixels, it stays perfectly sharp at any size, and because it is text it can sit directly inside the HTML file with no extra download.

`viewBox="0 0 28 28"` sets up a private coordinate system 28 units square; every coordinate inside is relative to that, so the whole drawing can be scaled to any size by CSS without touching the numbers. `fill="none"` sets the default to no fill, which each shape then overrides. `aria-hidden="true"` hides it from screen readers, correctly: the link already has a text label, and announcing the drawing too would be duplication.

Line 27 draws a 28 by 28 rounded square (`rx="8"` is the corner radius) in the brand blue. Line 28 draws the swoosh: `d` is a path description in a compact language where `M7 14` means "move to the point (7,14)" and the `c` and `s` commands that follow are curve instructions. It is written to be generated, not read, and nobody edits it by hand. Line 29 puts a white dot in the middle. Line 30 closes the drawing.

Line 31 is the wordmark. "digitalise" is plain text and "AI" is wrapped in a `<span>` labelled `logo-ai` so the stylesheet can colour those two letters blue. A `<span>` is the inline equivalent of a `<div>`: no meaning, no visual effect, purely a hook for styling.

Line 32 closes the logo link. Line 33 is the navigation's only button, a link to `#book`. Note it is a link and not a `<button>`, which is correct: it navigates somewhere rather than performing an action. It carries two classes, `btn` for the shared button shape and `btn-primary` for the filled blue variant. That two-class pattern repeats throughout the file and is explained in section 7.2.

Lines 34 and 35 close the containers.

Lines 36 to 57: the hero opens, and the first floating tile

```
36
37
38  <!-- ============================================
39       HERO
40  ============================================= -->
41  <section class="hero" aria-label="Hero">
42
43    <!-- animated background tiles -->
44    <div class="hero-bg" aria-hidden="true">
45
46      <!-- Tile 1: LinkedIn Video -->
47      <div class="tile tile-blue">
```

```
48        <div class="tile-thumb">
49          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-
        hidden="true">
50            <path stroke-linecap="round" stroke-linejoin="round" d="M15.75 10.5l4.72-4.72a.75.75
         0 011.28.53v11.38a.75.75 0 01-1.28.53l-4.72-4.72M4.5 18.75h9a2.25 2.25 0 002.25-2.25v-9a2
        .25 2.25 0 00-2.25-2.25h-9A2.25 2.25 0 002.25 7.5v9a2.25 2.25 0 002.25 2.25z"/>
51          </svg>
52        </div>
53        <div class="tile-meta">
54          <span class="tile-label">LinkedIn Video</span>
55          <span class="tile-metric">847K views</span>
56        </div>
57      </div>
```

Listing 3: `index.html`, lines 36–57

Lines 36 to 39 are a blank line and the second comment banner.

Line 40 opens the hero, the full-screen band at the top of the page. `<section>` is another semantic tag, meaning "a distinct part of the document".

Line 42 is a comment, and line 43 opens the container for the eight drifting cards. `aria-hidden="true"` on this container is the single most important accessibility decision in the file: it hides all eight tiles, and every number inside them, from screen readers. That is correct, because they are decoration. It is also the reason the invented numbers inside them are a presentation problem rather than an accessibility one.

Lines 46 to 57 are the first tile. `class="tile tile-blue"` again uses the two-class pattern: `tile` carries the shared card shape and the floating animation, `tile-blue` adds only the blue gradient behind the icon.

Inside, line 47 opens the thumbnail area, and lines 48 to 51 draw a video-camera icon. `stroke="currentColor"` on line 48 is worth understanding: it means "draw the outline in whatever text colour applies here", so the icon inherits its colour from CSS instead of hard-coding one. That single attribute is why one icon definition can appear in white on one background and blue on another.

Lines 53 to 56 are the tile's caption row: a label on the left and a metric on the right.

The tile metrics are invented

`847K views` is not a measurement. Neither are the other five numbers in these tiles: 234K impressions, 1.2M views, 512K views, 1.8M views and 19K reactions. Nothing in this repository or its history connects them to a real account or a real campaign, and they appeared fully formed in the first commit alongside the rest of the design.

They are decorative and hidden from assistive software, and a designer would call them realistic placeholder content, which is a normal thing to put in a mockup. But a sighted visitor cannot tell decoration from a claim, and reads them as results this agency has produced. That is worth raising specifically because the agency's main website was built to a stated standard of using no fabricated numbers.

Lines 58 to 70: the second tile, built from bars instead of an icor

```
58
59      <!-- Tile 2: Post tile -->
60      <div class="tile tile-orange">
61        <div class="tile-lines" style="padding:10px 0;">
62          <div class="tile-line"></div>
63          <div class="tile-line"></div>
64          <div class="tile-line"></div>
65        </div>
66        <div class="tile-meta">
67          <span class="tile-label">LinkedIn Post</span>
68          <span class="tile-metric">234K impr.</span>
```

```
69        </div>
70      </div>
```

Listing 4: `index.html`, lines 58–70

This tile represents a text post rather than a video, so instead of an icon it shows three grey bars standing in for lines of text. Each `<div class="tile-line">` is completely empty: it has no content at all, and exists only so that CSS can give it a height, a width and a rounded grey background. This is a standard technique for drawing an abstract shape without an image, and the widths (100%, 85%, 60%) are set in the stylesheet at lines 216 to 218 to suggest a paragraph ending mid-line.

An inline style that fights the stylesheet

`style="padding:10px 0;"` on line 61 sets the spacing directly on the element. The stylesheet also has a rule for this class, `.tile-lines { padding: 0; }` at line 209, and an inline style always wins over a stylesheet rule. So that rule never has any effect, in either of the two places the class is used. One of the two should go; the stylesheet rule is the dead one.

Jargon: Inline style

Appearance written directly on an element with a `style` attribute, rather than in the stylesheet. It overrides almost anything the stylesheet says, which makes it useful for one-offs and dangerous as a habit: rules written in one file stop being able to control what appears in the other.

Lines 71 to 80: the third tile, a statistic

```
71
72      <!-- Tile 3: Stats tile -->
73      <div class="tile" style="background:var(--s1);">
74        <div class="tile-stat-num">1.2M</div>
75        <div class="tile-stat-lbl">Views this month</div>
76        <div class="tile-meta" style="margin-top:12px;">
77          <span class="tile-label">AI Campaign</span>
78          <span class="tile-metric">+340%</span>
79        </div>
80      </div>
```

Listing 5: `index.html`, lines 71–80

This tile has no icon and no bars, just a large number, a caption and the usual meta row. Note `style="background:var(-s1);"` on line 73: an inline style that reads a custom property defined in the stylesheet. It works, and it is redundant, because the `.tile` rule at line 155 already sets exactly that background. Two more tiles (lines 96 and 121) repeat the same redundant inline background.

Lines 81 to 93: the fourth tile

```
81
82      <!-- Tile 4: Video -->
83      <div class="tile tile-teal">
84        <div class="tile-thumb">
85          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-
        hidden="true">
86            <path stroke-linecap="round" stroke-linejoin="round" d="M5.25 5.653c0-.856.917-1.398
         1.667-.986l11.54 6.348a1.125 1.125 0 010 1.971l-11.54 6.347a1.125 1.125 0 01-1.667-.985V5
        .653z"/>
87          </svg>
88        </div>
89        <div class="tile-meta">
90          <span class="tile-label">AI Video</span>
```

```
91          <span class="tile-metric">512K views</span>
92        </div>
93      </div>
```

Listing 6: `index.html`, lines 81–93

Structurally identical to tile 1, with two differences: the colour class is `tile-teal`, and the icon is a play triangle rather than a video camera. That same play-triangle path appears six more times in this file, once on each portfolio card. Nothing shares it: the same 180 characters of path data are repeated seven times in total, which is the cost of having no templating system. It is a real duplication, and it is also the correct trade for a project that deliberately has no build step.

Lines 94 to 105: the fifth tile, a status badge

```
94
95      <!-- Tile 5: Published badge -->
96      <div class="tile" style="background:var(--s1);">
97        <div style="display:flex;align-items:center;gap:8px;margin-bottom:10px;">
98          <svg viewBox="0 0 24 24" fill="none" stroke="#34d399" stroke-width="2" width="20"
        height="20" aria-hidden="true">
99            <path stroke-linecap="round" stroke-linejoin="round" d="M9 12.75L11.25 15 15 9.75M21
         12a9 9 0 11-18 0 9 9 0 0118 0z"/>
100         </svg>
101         <span style="font-family:var(--font-d);font-size:13px;font-weight:700;color:#34d399;">
        Published</span>
102       </div>
103       <div class="tile-stat-num" style="font-size:22px;">47 posts</div>
104       <div class="tile-stat-lbl">this week, zero manual work</div>
105     </div>
```

Listing 7: `index.html`, lines 94–105

This is the most heavily inline-styled element in the file: line 97 lays out a row, line 101 styles the word “Published”, and line 103 overrides a font size. None of it has a class, so none of it can be restyled from the stylesheet.

Note the green on lines 98 and 101 is written as the literal `#34d399`, even though the stylesheet defines exactly that colour as `-green` at line 15. Change the token and this tile would not follow. The same literal appears again at line 276 in the pipeline. This is the clearest illustration in the file of what inline styles cost: a colour system that only mostly applies.

Lines 106 to 118: the sixth tile

```
106
107     <!-- Tile 6: Violet video -->
108     <div class="tile tile-violet">
109       <div class="tile-thumb">
110         <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-
        hidden="true">
111           <path stroke-linecap="round" stroke-linejoin="round" d="M15.75 10.5l4.72-4.72a.75.75
         0 011.28.53v11.38a.75.75 0 01-1.28.53l-4.72-4.72M4.5 18.75h9a2.25 2.25 0 002.25-2.25v-9a2
        .25 2.25 0 00-2.25-2.25h-9A2.25 2.25 0 002.25 7.5v9a2.25 2.25 0 002.25 2.25z"/>
112         </svg>
113       </div>
114       <div class="tile-meta">
115         <span class="tile-label">AI Video</span>
116         <span class="tile-metric">1.8M views</span>
117       </div>
118     </div>
```

Listing 8: `index.html`, lines 106–118

Identical in structure to tile 1, and using the identical video-camera icon path, with a violet gradient and a different number. This is the third of four tiles built from the same template.

Lines 119 to 130: the seventh tile, a schedule

```
119
120     <!-- Tile 7: Schedule -->
121     <div class="tile" style="background:var(--s1);">
122       <div style="display:flex;align-items:center;gap:8px;margin-bottom:10px;">
123         <svg viewBox="0 0 24 24" fill="none" stroke="var(--accentH)" stroke-width="1.8" width=
        "18" height="18" aria-hidden="true">
124           <path stroke-linecap="round" stroke-linejoin="round" d="M6.75 3v2.25M17.25 3v2.25M3
        18.75V7.5a2.25 2.25 0 012.25-2.25h13.5A2.25 2.25 0 0121 7.5v11.25m-18 0A2.25 2.25 0 005.25
         21h13.5A2.25 2.25 0 0021 18.75m-18 0v-7.5A2.25 2.25 0 015.25 9h13.5A2.25 2.25 0 0121
        11.25v7.5"/>
125         </svg>
126         <span style="font-family:var(--font-d);font-size:12px;font-weight:600;color:var(--
        accentH);">Scheduled</span>
127       </div>
128       <div class="tile-stat-num" style="font-size:22px;">34 posts</div>
129       <div class="tile-stat-lbl">queued this week</div>
130     </div>
```

Listing 9: `index.html`, lines 119–130

The mirror image of tile 5: same layout, calendar icon instead of a tick, and "queued" instead of "published". Together the pair tells a small story in the background of the hero, that work is both finished and lined up. Note that here the colour *is* read from a token, `var(-accentH)` on lines 123 and 126, unlike tile 5 which used a literal. The two tiles were written minutes apart and are inconsistent with each other.

Lines 131 to 145: the eighth tile, and the end of the backdrop

```
131
132     <!-- Tile 8: Reactions -->
133     <div class="tile tile-blue">
134       <div class="tile-lines" style="padding:10px 0;">
135         <div class="tile-line"></div>
136         <div class="tile-line"></div>
137         <div class="tile-line" style="width:50%;"></div>
138       </div>
139       <div class="tile-meta">
140         <span class="tile-label">Viral Post</span>
141         <span class="tile-metric">19K ♡</span>
142       </div>
143     </div>
144
145   </div><!-- /hero-bg -->
```

Listing 10: `index.html`, lines 131–145

The last tile repeats the bar pattern from tile 2, overriding the third bar's width to 50% inline so the two text tiles do not look identical. Line 141 is where the heart character appears; as section 1.4 explained, this document prints it as a heart symbol.

Line 145 closes the tile container, with a trailing comment naming what was closed. That habit is worth copying: in a file with this much nesting, a bare `</div>` tells the reader nothing.

Why the tile order in the file is not the order on screen

The eight tiles are written top to bottom in the file, but they appear scattered around the hero. Nothing here places them. Each one is positioned by the stylesheet, using rules that count children: `.tile:nth-child(1)`, `(2)` and so on, at lines 162 to 169, each setting a corner and a delay before that tile starts drifting. **This means the order of these blocks in the HTML is load-bearing.** Move tile 5 above tile 3 in this file, without touching the CSS, and both tiles jump to new positions on screen and start animating at different times.

Lines 146 to 182: the overlay and the hero's actual content

```
146
147   <div class="hero-overlay" aria-hidden="true"></div>
148
149   <!-- hero content -->
150   <div class="hero-content">
151     <div class="hero-badge">
152       <span class="hero-badge-dot"></span>
153       Digitalise Agency
154     </div>
155
156     <h1 class="hero-h1">
157       AI systems built<br>
158       to grow while you <em>sleep.</em>
159     </h1>
160
161     <div class="hero-actions">
162       <a href="#book" class="btn btn-primary">
163         Book a call
164         <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor"
      stroke-width="2.5" aria-hidden="true">
165           <path stroke-linecap="round" stroke-linejoin="round" d="M13.5 4.5L21 12m0 0l-7.5 7.5
      M21 12H3"/>
166         </svg>
167       </a>
168       <a href="#portfolio" class="btn btn-ghost">See the work</a>
169     </div>
170
171     <div class="hero-social-proof">
172       <div class="avatars" aria-hidden="true">
173         <div class="avatar av1">K</div>
174         <div class="avatar av2">H</div>
175         <div class="avatar av3">M</div>
176         <div class="avatar av4">A</div>
177       </div>
178       <p class="hero-proof-text"><strong>17+ businesses</strong> growing with Digitalise AI</p
      >
179     </div>
180   </div>
181
182 </section>
```

Listing 11: `index.html`, lines 146–182

Line 147 is an empty `<div>` that exists purely to darken the middle of the hero so the headline stays readable over the drifting tiles. The stylesheet fills it with two stacked gradients (lines 232 to 243). An empty element used as a visual layer is a common and legitimate technique.

Lines 150 to 153 are the small pill above the headline. `hero-badge-dot` is another empty element, styled into a small circle that pulses.

Lines 156 to 159 are the headline, in an `<h1>`. A page should have exactly one `<h1>`, and this page does: it is the document's title as far as a screen reader or a search engine is concerned. `<br>` on line 157 forces a line break so the headline splits where the designer wants rather than where the window happens to be narrow.

`<em>` is used for colour, not for emphasis

Line 158 wraps "sleep." in `<em>`, and the stylesheet then removes the italics that tag normally implies and colours the word blue instead (lines 285 to 288). `<em>` means emphasis, and a screen reader may change its intonation for it. Using it as a colour hook works, and it slightly misrepresents the content to assistive software; a `<span>` with a class would have been the honest choice. This is a small point and the intent is plainly visual.

Lines 161 to 169 hold the two hero buttons. The first is the primary blue one, containing both text and an arrow icon; it points at `#book`. The second is the ghost variant, pointing at `#portfolio`. Both are ordinary links, and both are intercepted by the JavaScript at line 70 to scroll smoothly rather than jump.

Lines 171 to 179 are the social proof row: four overlapping circles bearing the letters K, H, M and A, and a line of text. The circles are `aria-hidden` because they carry no information.

The four avatar initials do not correspond to anything

K, H, M and A do not match the three clients named further down the page (Hi-Tec Bearings, DigiMarvixx, Synergy Event, whose own avatars use K, D and S). They are placeholder initials that were never replaced. The "17+ businesses" claim beside them matches the `data-target="17"` in the metrics band, so that number at least is internally consistent; whether it is externally true cannot be established from this repository.

Line 182 closes the hero section.

Lines 183 to 210: the metrics band

```
183
184
185 <!-- ============================================
186      METRICS BAND
187      UPDATE: replace data-target values with real numbers
188 ============================================= -->
189 <section class="metrics" aria-label="Results">
190   <div class="container">
191     <div class="metrics-grid stagger">
192       <div class="metric-item" style="--i:0">
193         <div class="metric-num" data-target="17" data-suffix="+">0</div>
194         <div class="metric-label">Projects Delivered</div>
195       </div>
196       <div class="metric-item" style="--i:1">
197         <div class="metric-num" data-target="1.6" data-suffix="M+">0</div>
198         <div class="metric-label">Organic Views for Clients</div>
199       </div>
200       <div class="metric-item" style="--i:2">
201         <div class="metric-num" data-target="10" data-suffix="K+">0</div>
202         <div class="metric-label">Users Served</div>
203       </div>
204       <div class="metric-item" style="--i:3">
205         <div class="metric-num" data-target="300" data-suffix="%">0</div>
206         <div class="metric-label">Avg. Traffic Growth</div>
207       </div>
208     </div>
209   </div>
210 </section>
```

Listing 12: `index.html`, lines 183–210

This is the first band that the JavaScript genuinely drives, and it introduces the two mechanisms the rest of the page reuses.

The first is the **data attribute**. Look at line 193: the element's visible content is the single character `0`, and the real number lives in `data-target="17"`, with `data-suffix="+"` beside it.

Jargon: Data attribute

Any attribute whose name starts with `data-`. HTML ignores it completely and displays nothing; it exists so that JavaScript can attach its own information to an element. `main.js` reads these two at lines 20 and 21 to know what number to count towards and what to print after it.

Why is the visible content `0` rather than `17`? Because that is what a visitor sees before the counter runs. It is also what a visitor sees *forever* if JavaScript is switched off or fails to load, which is the weakness of this pattern: the page would show four zeros and claim nothing. The robust alternative is to write the real number in the HTML and have the script count towards it from zero, which fails gracefully. This file chose the fragile form.

The second mechanism is the **stagger**. Line 191 gives the grid the class `stagger`, and each child carries `style="-i:0"`, `-i:1` and so on. That custom property is multiplied by 90 milliseconds in the stylesheet (line 90) to delay each child's fade-in, so the four items appear in sequence rather than together. Setting a custom property inline is the cleanest way to pass a per-element number to CSS, and it is used four times in this file.

Note that this band still carries an `UPDATE` comment on line 187 telling the reader to replace the values with real numbers, even though commit `2c66a7a` claimed to have done exactly that. See section 14.

Lines 211 to 235: the pipeline section and its first node

```
211
212
213 <!-- ================================================
214      HOW IT WORKS (PIPELINE)
215 ================================================ -->
216 <section class="pipeline-section" aria-label="How it works">
217   <div class="container">
218     <div class="reveal">
219       <p class="section-kicker">How it works</p>
220       <h2 class="section-title">From idea to live<br>in one click.</h2>
221     </div>
222
223     <div class="pipeline" aria-label="Content production pipeline">
224
225       <!-- Node 1 -->
226       <div class="pipeline-node">
227         <div class="pipeline-icon">
228           <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-
      hidden="true">
229             <path stroke-linecap="round" stroke-linejoin="round" d="M12 18v-5.25m0 0a6.01 6.01
       0 001.5-.189m-1.5.189a6.01 6.01 0 01-1.5-.189m3.75 7.478a12.06 12.06 0 01-4.5 0m3.75
      2.355a14.998 14.998 0 01-3 0M14.25 18v-.192c0-.983.658-1.823 1.508-2.316a7.5 7.5 0
      10-7.517 0c.85.493 1.509 1.333 1.509 2.316V18"/>
230           </svg>
231         </div>
232         <span class="pipeline-lbl">Your Idea</span>
233       </div>
234
235       <div class="pipeline-connector" aria-hidden="true"></div>
```

Listing 13: `index.html`, lines 211–235

Lines 218 to 221 establish a pattern used by all four remaining sections: a wrapper with the class `reveal`, containing a small uppercase "kicker" line and a large heading. The `reveal` class is what the JavaScript watches for at line 56, fading the pair upward into place when they scroll into view.

Line 220 is an `<h2>`. Heading levels form the document's outline: the single `<h1>` in the hero, then an `<h2>` per major section. Getting that order right is how a screen-reader user navigates a long page, and this file gets it right throughout.

Line 223 opens the pipeline, and line 235 is the first connector: an entirely empty element that the stylesheet turns into a short line between two nodes, and then fills with a blue gradient when the section becomes visible. The nodes and connectors alternate, which is why the stylesheet targets odd children for nodes and even children for connectors (lines 400 to 426).

Lines 236 to 285: the remaining four pipeline nodes

```
      <!-- Node 2 -->
      <div class="pipeline-node">
        <div class="pipeline-icon">
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-hidden="true">
            <path stroke-linecap="round" stroke-linejoin="round" d="M16.862 4.487l1.687-1.688a1.875 1.875 0 112.652 2.652L10.582 16.07a4.5 4.5 0 01-1.897 1.13L6 18l.8-2.685a4.5 4.5 0 011.13-1.897l8.932-8.931zm0 0L19.5 7.125M18 14v4.75A2.25 2.25 0 0115.75 21H5.25A2.25 2.25 0 013 18.75V8.25A2.25 2.25 0 015.25 6H10"/>
          </svg>
        </div>
        <span class="pipeline-lbl">AI Script</span>
      </div>

      <div class="pipeline-connector" aria-hidden="true"></div>

      <!-- Node 3 -->
      <div class="pipeline-node">
        <div class="pipeline-icon">
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-hidden="true">
            <path stroke-linecap="round" stroke-linejoin="round" d="M15.75 10.5l4.72-4.72a.75.75 0 011.28.53v11.38a.75.75 0 01-1.28.53l-4.72-4.72M4.5 18.75h9a2.25 2.25 0 002.25-2.25v-9a2.25 2.25 0 00-2.25-2.25h-9A2.25 2.25 0 002.25 7.5v9a2.25 2.25 0 002.25 2.25z"/>
          </svg>
        </div>
        <span class="pipeline-lbl">AI Video</span>
      </div>

      <div class="pipeline-connector" aria-hidden="true"></div>

      <!-- Node 4 -->
      <div class="pipeline-node">
        <div class="pipeline-icon">
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-hidden="true">
            <path stroke-linecap="round" stroke-linejoin="round" d="M6 12L3.269 3.126A59.768 59.768 0 0121.485 12 59.77 59.77 0 013.27 20.876L5.999 12zm0 0h7.5"/>
          </svg>
        </div>
        <span class="pipeline-lbl">Auto-Post</span>
      </div>

      <div class="pipeline-connector" aria-hidden="true"></div>

      <!-- Node 5 -->
      <div class="pipeline-node">
        <div class="pipeline-icon" style="background:rgba(52,211,153,.08);border-color:rgba(52,211,153,.25);">
          <svg viewBox="0 0 24 24" fill="none" stroke="#34d399" stroke-width="1.8" aria-hidden="true">
            <path stroke-linecap="round" stroke-linejoin="round" d="M9 12.75L11.25 15 15 9.75M21 12a9 9 0 11-18 0 9 9 0 0118 0z"/>
          </svg>
        </div>
        <span class="pipeline-lbl" style="color:#34d399;">Live</span>
      </div>

    </div><!-- /pipeline -->
  </div>
</section>
```

Listing 14: `index.html`, lines 236–285

Four more nodes, each built to the identical shape: a container, an icon box, an SVG, and a label. The icons in order are a lightbulb (an idea), a pencil on a document (a script), a video camera (a video, reusing the same path as tile 1 and tile 6), and a paper plane (posting).

Node 5 is the only one that differs, and it differs three times over: it overrides its background and border colour inline on line 275, hard-codes the same green in the SVG stroke on line 276, and colours its label inline on line 280. Everything is written to make the final step green rather than blue, and none of it uses the `-green` token that exists for exactly this purpose. The same criticism as tile 5 applies, and this is the fourth and last place the literal `#34d399` appears.

The five steps read as a claim about the product: an idea becomes a script, becomes a video, gets posted automatically, and is live. That is the same pipeline the Gantry project in this workspace actually implements, so the page is advertising a real capability rather than an invented one, though nothing in this repository connects the two.

Lines 286 to 318: the portfolio section and its first card

```
286
287
288 <!-- ============================================
289      PORTFOLIO
290      UPDATE: replace gradient .pt* classes with real video poster images.
291      Add poster="path/to/thumb.jpg" to each <video> or swap divs for <img>.
292 ============================================= -->
293 <section class="portfolio-section" id="portfolio" aria-label="Work portfolio">
294   <div class="container">
295     <div class="reveal" style="margin-bottom:48px;">
296       <p class="section-kicker">The work</p>
297       <h2 class="section-title">Real content.<br>Real results.</h2>
298     </div>
299
300     <div class="portfolio-grid stagger">
301
302       <div class="port-card" style="--i:0">
303         <div class="port-thumb pt1">
304           <div class="port-badge">847K views</div>
305           <div class="port-thumb-overlay">
306             <div class="port-play">
307               <svg viewBox="0 0 24 24" fill="white" aria-hidden="true"><path d="M5.25 5.653c0
      -.856.917-1.398 1.667-.986l11.54 6.348a1.125 1.125 0 010 1.971l-11.54 6.347a1.125 1.125 0
      01-1.667-.985V5.653z"/></svg>
308             </div>
309           </div>
310         </div>
311         <div class="port-info">
312           <div class="port-platform">
313             <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-
      hidden="true"><path stroke-linecap="round" stroke-linejoin="round" d="M13.5 4.5L21 12m0 0l
      -7.5 7.5M21 12H3"/></svg>
314             E-commerce
315           </div>
316           <div class="port-title">Hi-Tec Bearings: 5,000+ SKUs, 300% organic traffic growth</
      div>
317         </div>
318       </div>
```

Listing 15: `index.html`, lines 286–318

Line 293 carries `id="portfolio"`, which is what makes the hero's "See the work" button work. An `id` is a unique name for one element, and a link to `#portfolio` scrolls to whatever carries that name.

Jargon: id, and how it differs from class

An `id` names exactly one element on the page and must be unique. A `class` labels any number of them. Use an `id` when something needs to be linked to or found individually; use a `class` when you want to style a group. This page has exactly two ids, `portfolio` and `book`, and both exist to be jumped to.

Each card is built from two halves. The top half, `port-thumb`, is where a real video thumbnail would go; today it is a gradient supplied by the class `pt1`, defined at line 504 of the stylesheet. Over it sit a badge in the corner and an overlay containing a play button, which the stylesheet keeps invisible until the mouse is over the card. The bottom half, `port-info`, holds a small platform label with an arrow icon and the project title.

The six projects named across these cards are Hi-Tec Bearings, DigiMarvixx, Synergy Event, IBA GPT, Prestige Sleep and IBA CED. Unlike the hero tiles, these are specific, named and checkable, which is a meaningfully different kind of claim.

The play buttons promise videos that do not exist

Every card has a play button that appears on hover, and none of them plays anything: the card is not a link and has no click handler. The stylesheet even sets `cursor: pointer` on it (line 465), so the pointer changes to a hand, promising an action that never happens. `ASSETS.md` confirms the intent was real video, and the video was never supplied. As it stands the hover behaviour is a small deception, and the cheapest honest fix is to remove the play overlay until the videos exist.

Lines 319 to 354: portfolio cards two and three

```
319
320       <div class="port-card" style="--i:1">
321         <div class="port-thumb pt2">
322           <div class="port-badge">3x leads</div>
323           <div class="port-thumb-overlay">
324             <div class="port-play">
325               <svg viewBox="0 0 24 24" fill="white" aria-hidden="true"><path d="M5.25 5.653c0
      -.856.917-1.398 1.667-.986l11.54 6.348a1.125 1.125 0 010 1.971l-11.54 6.347a1.125 1.125 0
      01-1.667-.985V5.653z"/></svg>
326             </div>
327           </div>
328         </div>
329         <div class="port-info">
330           <div class="port-platform">
331             <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-
      hidden="true"><path stroke-linecap="round" stroke-linejoin="round" d="M13.5 4.5L21 12m0 0l
      -7.5 7.5M21 12H3"/></svg>
332             Web + WhatsApp
333           </div>
334           <div class="port-title">DigiMarvixx: 3x monthly WhatsApp leads, 60% mobile traffic</
      div>
335         </div>
336       </div>
337
338       <div class="port-card" style="--i:2">
339         <div class="port-thumb pt3">
340           <div class="port-badge">1.2s load</div>
341           <div class="port-thumb-overlay">
342             <div class="port-play">
343               <svg viewBox="0 0 24 24" fill="white" aria-hidden="true"><path d="M5.25 5.653c0
      -.856.917-1.398 1.667-.986l11.54 6.348a1.125 1.125 0 010 1.971l-11.54 6.347a1.125 1.125 0
      01-1.667-.985V5.653z"/></svg>
344             </div>
```

```
345           </div>
346         </div>
347         <div class="port-info">
348           <div class="port-platform">
349             <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-
        hidden="true"><path stroke-linecap="round" stroke-linejoin="round" d="M13.5 4.5L21 12m0 0l
        -7.5 7.5M21 12H3"/></svg>
350             Events
351           </div>
352           <div class="port-title">Synergy Event: 800 participants, 98/100 Lighthouse score</
        div>
353         </div>
354       </div>
```

Listing 16: `index.html`, lines 319–354

Structurally identical to card one, differing only in the stagger index (`-i:1` and `-i:2`), the gradient class (`pt2`, `pt3`), the badge text, the platform label and the title. Note that the same arrow icon is used as the "platform" icon on every card regardless of what the platform actually is, so it is decorative rather than informative.

"98/100 Lighthouse score" on line 352 is a specific, verifiable claim. Lighthouse is Google's page-quality auditing tool, and 98 out of 100 is a strong result.

Lines 355 to 412: portfolio cards four, five and six

```
355
356       <div class="port-card" style="--i:3">
357         <div class="port-thumb pt4">
358           <div class="port-badge">AI chat</div>
359           <div class="port-thumb-overlay">
360             <div class="port-play">
361               <svg viewBox="0 0 24 24" fill="white" aria-hidden="true"><path d="M5.25 5.653c0
        -.856.917-1.398 1.667-.986l11.54 6.348a1.125 1.125 0 010 1.971l-11.54 6.347a1.125 1.125 0
        01-1.667-.985V5.653z"/></svg>
362             </div>
363           </div>
364         </div>
365         <div class="port-info">
366           <div class="port-platform">
367             <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-
        hidden="true"><path stroke-linecap="round" stroke-linejoin="round" d="M13.5 4.5L21 12m0 0l
        -7.5 7.5M21 12H3"/></svg>
368             AI Assistant
369           </div>
370           <div class="port-title">IBA GPT: AI assistant answering questions about IBA
        university</div>
371         </div>
372       </div>
373
374       <div class="port-card" style="--i:4">
375         <div class="port-thumb pt5">
376           <div class="port-badge">24/7 sales</div>
377           <div class="port-thumb-overlay">
378             <div class="port-play">
379               <svg viewBox="0 0 24 24" fill="white" aria-hidden="true"><path d="M5.25 5.653c0
        -.856.917-1.398 1.667-.986l11.54 6.348a1.125 1.125 0 010 1.971l-11.54 6.347a1.125 1.125 0
        01-1.667-.985V5.653z"/></svg>
380             </div>
381           </div>
382         </div>
383         <div class="port-info">
384           <div class="port-platform">
385             <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-
        hidden="true"><path stroke-linecap="round" stroke-linejoin="round" d="M13.5 4.5L21 12m0 0l
```

```
      -7.5 7.5M21 12H3"/></svg>
386           Health + E-commerce
387         </div>
388         <div class="port-title">Prestige Sleep: clinic booking and product shop in one site<
      /div>
389       </div>
390     </div>
391
392     <div class="port-card" style="--i:5">
393       <div class="port-thumb pt6">
394         <div class="port-badge">4 dashboards</div>
395         <div class="port-thumb-overlay">
396           <div class="port-play">
397             <svg viewBox="0 0 24 24" fill="white" aria-hidden="true"><path d="M5.25 5.653c0
      -.856.917-1.398 1.667-.986l11.54 6.348a1.125 1.125 0 010 1.971l-11.54 6.347a1.125 1.125 0
      01-1.667-.985V5.653z"/></svg>
398           </div>
399         </div>
400       </div>
401       <div class="port-info">
402         <div class="port-platform">
403           <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-
      hidden="true"><path stroke-linecap="round" stroke-linejoin="round" d="M13.5 4.5L21 12m0 0l
      -7.5 7.5M21 12H3"/></svg>
404           Analytics
405         </div>
406         <div class="port-title">IBA CED: analytics platform tracking entrepreneurship
      outcomes</div>
407       </div>
408     </div>
409
410   </div><!-- /portfolio-grid -->
411  </div>
412 </section>
```

Listing 17: `index.html`, lines 355–412

The final three cards complete the set of six, following the same template with indices 3, 4 and 5 and gradients `pt4` through `pt6`. Two of the six projects are for the same institution (IBA GPT and the analytics platform), which is the only repetition among them.

Line 410 closes the grid with a naming comment, and lines 411 and 412 close the container and the section.

Lines 413 to 472: the client proof band

```
413
414
415 <!-- ================================================
416      CLIENT PROOF
417      UPDATE: replace initials and numbers with real client data
418 ================================================ -->
419 <section class="proof-section" aria-label="Client results">
420   <div class="container">
421     <div class="reveal" style="margin-bottom:48px;">
422       <p class="section-kicker">Client results</p>
423       <h2 class="section-title">Numbers that<br>speak for themselves.</h2>
424     </div>
425
426     <div class="proof-grid stagger">
427
428       <div class="proof-card" style="--i:0">
429         <div class="proof-header">
430           <div class="proof-avatar pa1">K</div>
431           <div>
```

```
            <div class="proof-name">Hi-Tec Bearings</div>
            <div class="proof-company">Industrial E-commerce</div>
          </div>
        </div>
        <div class="proof-metric">
          <span class="proof-num">300%</span>
          <span class="proof-unit">organic traffic growth</span>
        </div>
      </div>

      <div class="proof-card" style="--i:1">
        <div class="proof-header">
          <div class="proof-avatar pa2">D</div>
          <div>
            <div class="proof-name">DigiMarvixx</div>
            <div class="proof-company">Digital Marketing</div>
          </div>
        </div>
        <div class="proof-metric">
          <span class="proof-num">3x</span>
          <span class="proof-unit">monthly WhatsApp leads</span>
        </div>
      </div>

      <div class="proof-card" style="--i:2">
        <div class="proof-header">
          <div class="proof-avatar pa3">S</div>
          <div>
            <div class="proof-name">Synergy Event</div>
            <div class="proof-company">Events & Conferences</div>
          </div>
        </div>
        <div class="proof-metric">
          <span class="proof-num">800</span>
          <span class="proof-unit">participants, 98/100 Lighthouse</span>
        </div>
      </div>

    </div>
  </div>
</section>
```

Listing 18: `index.html`, lines 413–472

Three cards, each with a coloured circle bearing an initial, a client name, an industry, and one headline number. The numbers repeat what the portfolio cards already said about the same three clients, which is deliberate: repetition is how a landing page makes a claim stick.

Note line 430: the avatar for Hi-Tec Bearings shows the letter **K**, not H. That is almost certainly a leftover from the placeholder set in the hero (K, H, M, A), and it is the clearest surviving trace of the `UPDATE` comment on line 417 having been only partly acted on.

Line 461 contains `Events & Conferences`. The ampersand is written as a bare character rather than as the escape `&`. Browsers forgive this, and it is technically invalid: `&` begins a character reference in HTML. It works here only because what follows is a space, so no valid reference can be formed. It is worth knowing as a habit to avoid rather than a bug to fix.

Jargon: Character reference, or entity

A way of writing characters that would otherwise be interpreted as markup: `&` for an ampersand, `<` for a less-than sign. They exist because those characters have structural meaning in HTML.

Lines 473 to 549: the services grid

```
473
474
475 <!-- ============================================
476      SERVICES
477 ============================================= -->
478 <section class="services-section" aria-label="Services">
479   <div class="container">
480     <div class="reveal" style="margin-bottom:48px;">
481       <p class="section-kicker">What we do</p>
482       <h2 class="section-title">Everything your content<br>operation needs.</h2>
483     </div>
484
485     <div class="services-grid stagger">
486
487       <div class="service-item" style="--i:0">
488         <div class="service-icon">
489           <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-
      hidden="true">
490             <path stroke-linecap="round" stroke-linejoin="round" d="M15.75 10.5l4.72-4.72a
      .75.75 0 011.28.53v11.38a.75.75 0 01-1.28.53l-4.72-4.72M4.5 18.75h9a2.25 2.25 0
      002.25-2.25v-9a2.25 2.25 0 00-2.25-2.25h-9A2.25 2.25 0 002.25 7.5v9a2.25 2.25 0 002.25
      2.25z"/>
491           </svg>
492         </div>
493         <div class="service-name">AI Personal Brand</div>
494         <span class="service-tag">Your voice, published</span>
495       </div>
```

Listing 19: `index.html`, lines 473–495

The services grid follows the established shape: a revealed heading pair, then a staggered grid. Each of the six items is an icon box, a name, and a small uppercase tag line. The first item's icon is, once again, the same video-camera path from tile 1.

The remaining five items are structurally identical and are quoted in full below, since the coverage rule admits no "and so on".

```
496
497       <div class="service-item" style="--i:1">
498         <div class="service-icon">
499           <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-
      hidden="true">
500             <path stroke-linecap="round" stroke-linejoin="round" d="M21 21l-5.197-5.197m0 0A7
      .5 7.5 0 105.196 5.196a7.5 7.5 0 0010.607 10.607z"/>
501           </svg>
502         </div>
503         <div class="service-name">AI SEO & Content</div>
504         <span class="service-tag">Rank for what they ask</span>
505       </div>
506
507       <div class="service-item" style="--i:2">
508         <div class="service-icon">
509           <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-
      hidden="true">
510             <path stroke-linecap="round" stroke-linejoin="round" d="M8.625 12a.375.375 0
      11-.75 0 .375.375 0 01.75 0zm0 0H8.25m4.125 0a.375.375 0 11-.75 0 .375.375 0 01.75 0zm0 0
      H12m4.125 0a.375.375 0 11-.75 0 .375.375 0 01.75 0zm0 0h-.375M21 12c0 4.556-4.03 8.25-9
      8.25a9.764 9.764 0 01-2.555-.337A5.972 5.972 0 015.41 20.97a5.969 5.969 0 01-.474-.065
      4.48 4.48 0 00.978-2.025c.09-.457-.133-.901-.467-1.226C3.93 16.178 3 14.189 3 12c0-4.556
      4.03-8.25 9-8.25s9 3.694 9 8.25z"/>
511           </svg>
512         </div>
513         <div class="service-name">AI Chat Agent</div>
514         <span class="service-tag">Reply in seconds</span>
```

```
515       </div>
516
517       <div class="service-item" style="--i:3">
518         <div class="service-icon">
519           <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-
      hidden="true">
520             <path stroke-linecap="round" stroke-linejoin="round" d="M2.25 6.75c0 8.284 6.716
      15 15 15h2.25a2.25 2.25 0 002.25-2.25v-1.372c0-.516-.351-.966-.852-1.091l-4.423-1.106c
      -.44-.11-.902.055-1.173.417l-.97 1.293c-.282.376-.769.542-1.21.38a12.035 12.035 0
      01-7.143-7.143c-.162-.441.004-.928.38-1.21l1.293-.97c.363-.271.527-.734.417-1.173L6.963
      3.102a1.125 1.125 0 00-1.091-.852H4.5A2.25 2.25 0 002.25 4.5v2.25z"/>
521           </svg>
522         </div>
523         <div class="service-name">AI Voice Agent</div>
524         <span class="service-tag">Answer calls 24/7</span>
525       </div>
526
527       <div class="service-item" style="--i:4">
528         <div class="service-icon">
529           <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-
      hidden="true">
530             <path stroke-linecap="round" stroke-linejoin="round" d="M21.75 6.75v10.5a2.25 2.25
       0 01-2.25 2.25h-15a2.25 2.25 0 01-2.25-2.25V6.75m19.5 0A2.25 2.25 0 0019.5 4.5h-15a2.25
      2.25 0 00-2.25 2.25m19.5 0v.243a2.25 2.25 0 01-1.07 1.916l-7.5 4.615a2.25 2.25 0 01-2.36 0
      L3.32 8.91a2.25 2.25 0 01-1.07-1.916V6.75"/>
531           </svg>
532         </div>
533         <div class="service-name">AI Email Outreach</div>
534         <span class="service-tag">Follow up until they respond</span>
535       </div>
536
537       <div class="service-item" style="--i:5">
538         <div class="service-icon">
539           <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" aria-
      hidden="true">
540             <path stroke-linecap="round" stroke-linejoin="round" d="M9.53 16.122a3 3 0 00-5.78
       1.128 2.25 2.25 0 01-2.4 2.245 4.5 4.5 0 008.4-2.245c0-.399-.078-.78-.22-1.128zm0 0a15
      .998 15.998 0 003.388-1.62m-5.043-.025a15.994 15.994 0 011.622-3.395m3.42 3.42a15.995
      15.995 0 004.764-4.648l3.876-5.814a1.151 1.151 0 00-1.597-1.597L14.146 6.32a15.996 15.996
      0 00-4.649 4.763m3.42 3.42a6.776 6.776 0 00-3.42-3.42"/>
541           </svg>
542         </div>
543         <div class="service-name">AI Ad Creatives</div>
544         <span class="service-tag">On-brand at volume</span>
545       </div>
546
547     </div>
548   </div>
549 </section>
```

Listing 20: `index.html`, lines 496–549: services two to six

The six services are AI Personal Brand, AI SEO and Content, AI Chat Agent, AI Voice Agent, AI Email Outreach and AI Ad Creatives. Their icons in order are a video camera, a magnifying glass, a speech bubble, a telephone handset, an envelope and a paintbrush, each one an obvious visual match for its service, which is why they need no labels of their own.

Line 503 contains another unescaped ampersand in “AI SEO & Content”, the same point as line 461.

Note the grid gets its dividing lines by a trick rather than by borders: the stylesheet gives the grid a 2-pixel gap with the border colour as the grid’s own background, then gives each cell the page background (lines 608 to 618). The gaps show the layer beneath, producing hairlines with no border rules at all. It is elegant, and it is the kind of thing that looks broken if you change one

of the two backgrounds without the other.

Lines 550 to 580: the booking panel

```
550
551
552 <!-- ============================================
553      BOOK A CALL
554      UPDATE: replace YOUR_CAL_LINK with Salman's Cal.com or Calendly URL.
555      For inline Cal.com embed, swap the anchor tag for the Cal embed snippet.
556 ============================================= -->
557 <section class="book-section" id="book" aria-label="Book a call">
558   <div class="container">
559     <div class="book-wrap reveal">
560       <p class="book-eyebrow">Ready to grow?</p>
561       <h2 class="book-title">30 minutes.<br>No slides.</h2>
562       <p class="book-sub">Pick a time. We talk about your content goals.</p>
563       <div class="book-cta-row">
564         <!-- UPDATE: replace href with your Cal.com or Calendly link -->
565         <a href="https://cal.com/davonex/30min" target="_blank" rel="noopener" class="btn btn-
      primary" style="padding:16px 36px;font-size:17px;">
566           Book a call
567           <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor"
      stroke-width="2.5" aria-hidden="true">
568             <path stroke-linecap="round" stroke-linejoin="round" d="M13.5 4.5L21 12m0 0l-7.5
      7.5M21 12H3"/>
569           </svg>
570         </a>
571       </div>
572       <p class="book-detail">
573         <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" aria-
      hidden="true">
574           <path stroke-linecap="round" stroke-linejoin="round" d="M12 6v6h4.5m4.5 0a9 9 0
      11-18 0 9 9 0 0118 0z"/>
575         </svg>
576         Free · <span>30 minutes · Zoom</span>
577       </p>
578     </div>
579   </div>
580 </section>
```

Listing 21: `index.html`, lines 550–580

This is the destination of every "Book a call" button on the page, via `id="book"` on line 557. It is the point of the whole document.

Line 559 combines two classes on one element, `book-wrap` for the panel's appearance and `reveal` so the JavaScript fades it in. That is the only place in the file where a layout class and a behaviour class share an element.

Line 565 is the actual booking link, and three of its attributes matter. `target="_blank"` opens it in a new tab, so the visitor does not lose the page. `rel="noopener"` is a security measure that must accompany it: without it, the newly opened page is handed a reference back to this one and can navigate it somewhere else, a trick known as tab-nabbing. Its presence here is correct and is the kind of detail that is usually forgotten.

The booking link is a placeholder that was never replaced

`cal.com/davonex/30min` is the calendar of Davonex, a different agency project, not a Digitalise AI calendar. Commit `4dffc12` on 2026-07-28 set this deliberately, and the same placeholder calendar is recorded as an outstanding item on the agency's main website too, so it is a known shared loose end rather than a slip here.

Two things around it are worse than the link itself. The comment on line 554 still tells

the reader to search for `YOUR_CAL_LINK`, which no longer exists anywhere in the file, and `ASSETS.md` repeats that same stale instruction. Somebody following the written instructions would search, find nothing, and conclude the work was done.

Lines 572 to 577 are the reassurance line under the button: a clock icon, then "Free", then the duration and the platform. The middle dots here are the U+00B7 character discussed in section 1.4, and they render exactly.

Lines 581 to 597: the footer, and the script tag

```
581
582
583 <!-- ============================================
584      FOOTER
585 ============================================= -->
586 <footer>
587   <div class="container">
588     <p>
589       <a href="/">digitalise.agency</a>  · 
590       <a href="mailto:hello@digitalise.agency">hello@digitalise.agency</a>
591     </p>
592   </div>
593 </footer>
594
595 <script src="/js/main.js"></script>
596 </body>
597 </html>
```

Listing 22: `index.html`, lines 581–597

The footer is one line of text: a link home and an email address, separated by a middle dot padded with ` ` on each side.

Jargon: Non-breaking space

` ` is a space that a browser will not use as a place to wrap a line, and which is not collapsed when several spaces sit together. Here it does the second job: ordinary spaces around the dot would collapse to one, and these keep the separator visually padded.

Line 589 links to `/`, meaning the site's own home page. The visible text says `digitalise.agency`, which is a different site from `ai.digitalise.agency`. So the footer link appears to promise the main agency site and in fact reloads this one. On the live site that would be a small misdirection; here it is another loose end.

Line 590 is a `mailto:` link, which opens the visitor's email program with the address filled in.

Line 595 loads the JavaScript, and **where it sits is the whole point**. It is the last thing before `</body>`, so by the time the browser reaches it, every element on the page already exists. `main.js` begins by searching for elements immediately, with no wrapper telling it to wait, and that only works because of this placement. Move this one line up into the `<head>` and the script would run against an empty document, find nothing, and the page would quietly lose all three of its behaviours with no error message.

Lines 596 and 597 close the body and the document.

What it connects to

This file names: `new-design-reference/css/site.css` (line 16), `new-design-reference/js/main.js` (line 595), Google Fonts (lines 13 to 15), and cal.com (line 565).

This file is named by: nothing inside the repository. The deploy workflow copies the whole folder rather than referring to files individually, and `netlify.toml` publishes the folder as a whole. `ASSETS.md` discusses it at length but cannot affect it.

Internal links: the hero and navigation both point at `#book` (lines 33 and 162), and the hero also points at `#portfolio` (line 168). Those two ids, on lines 557 and 293, are the only link targets in the file.

How it breaks

- **Rename a class here without renaming it in the stylesheet** and that element loses its appearance silently. Nothing warns you.
- **Move the `<script>` tag into the `<head>`** and all three behaviours stop working, with no error in the console. The metrics band would then permanently display four zeros.
- **Reorder the eight hero tiles** and they jump to different positions, because the stylesheet positions them by their number, not by their class.
- **Open the file directly off a disk** and it renders unstyled, because of the leading slash on line 16. This looks like catastrophic breakage and is not. See section 11.
- **Disable JavaScript** and the page is fully readable but static: the metrics show zeros, nothing fades in, and the anchor links jump instead of gliding. Only the metrics are a real loss.
- **Delete the id on line 557** and every "Book a call" button on the page silently does nothing, because `main.js` returns early when it cannot find a link's target.

Honest findings for this file

Collected here, each already argued above: the invented tile metrics (lines 55, 68, 74, 91, 116, 141); the missing `og:image` and favicon; the mismatched avatar initial on line 429; five stale `UPDATE` comments (lines 187, 290, 417, 554, 564); the placeholder booking link on line 565; two unescaped ampersands (lines 461 and 503); `<em>` used as a colour hook on line 158; play buttons that play nothing; a footer link whose text and destination disagree; the literal `#34d399` used four times where a token exists; and one inline padding that permanently overrides a stylesheet rule.

7.2 `new-design-reference/css/site.css`

CSS, 768 lines. Every visual decision on the page: the colours, the typefaces, the spacing, the layout, the movement, and how all of it rearranges on a narrow screen. It is the longest file in the repository and, in the judgement of this document, the most valuable thing in it.

Where this came from: new-design-reference/css/site.css

Hand-written, and it says where its ideas come from in its own second line: `Design tokens mirror the Digitalise Agency system (tokens.css in Gantry).` That is a direct statement by the author that the colour and spacing names were taken from another project in this workspace, `gantry-digitalise`, which is strong evidence of a deliberate shared design language rather than a coincidence.

The file arrived complete in the first commit, `0d77d51`, and has not been edited since except by being moved. No generator banner, no vendor prefix block of the sort a preprocessor emits, and no minified section: it is written to be read. **Confidence: certain for hand-written, high for the token lineage, which rests on the author's own comment rather than on a file comparison.**

Why it exists

Without this file the page still contains every word it needs, and looks like a 1994 document: black serif text on white, everything in one column, no colour, no layout. Nothing would be broken and nothing would be persuasive. All eight hero tiles would appear stacked in a list, since it is CSS and not HTML that scatters them.

Line by line

Lines 1 to 25: the header, and the token block

```
1  /* ai.digitalise.agency · site.css
2     Design tokens mirror the Digitalise Agency system (tokens.css in Gantry). */
3
4  /* --- TOKENS --- */
5  :root {
6    --s0:    #05070d;
7    --s1:    #0d1220;
8    --s2:    #151c2e;
9    --ink1:  #f4f7fc;
10   --ink2:  #c3ccdd;
11   --ink3:  #93a0b8;
12   --accent:  #1967ff;
13   --accentH: #3f82ff;
14   --accent2: #ff6719;
15   --green:   #34d399;
16   --border:  #222b40;
17   --borderS: #1a2233;
18   --font-d: 'Space Grotesk', ui-sans-serif, system-ui, sans-serif;
19   --font-b: 'Inter', ui-sans-serif, system-ui, sans-serif;
20   --r:   16px;
21   --rsm: 10px;
22   --ease: cubic-bezier(.22,1,.36,1);
23   --maxw: 1320px;
24   --shadow: 0 8px 40px rgba(0,0,0,.45);
25 }
```

Listing 23: `css/site.css`, lines 1–25

Lines 1 and 2 are a comment. In CSS, anything between `/*` and `*/` is ignored, and unlike HTML there is only this one comment form. These two lines carry the file's entire provenance, which is why they matter out of all proportion to their length.

Line 4 is the first of eleven section banners drawn with box-drawing characters. As with the HTML, they are the author's navigation aid.

Line 5 opens the token block. `:root` is a selector that matches the document's outermost element, which is `<html>`. Custom properties defined there are inherited by every element on the page, which is precisely why token blocks are written against `:root`: it is the one place a definition reaches everything.

The twenty tokens fall into six groups, and understanding the naming scheme is most of understanding the file:

Tokens	Line	What the group is for
`-s0` to `-s2`	6–8	Surfaces, darkest to lightest. `s0` is the page, `s1` is a card, `s2` is a detail inside a card
`-ink1` to `-ink3`	9–11	Text, brightest to dimmest. `ink1` is a heading, `ink2` is body text, `ink3` is a caption
`-accent`, `-accentH`, `-accent2`	12–14	The brand blue, its hover variant, and an orange
`-green`	15	The one success colour
`-border`, `-borderS`	16–17	A visible hairline, and a subtler one
`-font-d`, `-font-b`	18–19	The display typeface and the body typeface
`-r`, `-rsm`, `-ease`, `-maxw`, `-shadow`	20–24	Corner radius, small radius, motion curve, content width, drop shadow

Table 3: The token vocabulary. The numeric suffix always means the same thing: 0 is furthest back or brightest, and the number rises as you come forward or dim down.

Why a numbered scale beats named colours

Calling a colour `-dark-navy` ties the name to how it looks. Calling it `-s1` ties the name to what it is *for*: the surface one step up from the page. Redesign the site in a light theme and `-s1` still means the same thing while `-dark-navy` becomes a lie. This is the single most transferable idea in this repository, and it is why the author borrowed the scheme from another project rather than inventing one.

Line 18 defines the display typeface as Space Grotesk with three fallbacks after it. A browser reads that list left to right and uses the first one it can find. `ui-sans-serif` means the interface font of the operating system, `system-ui` is the same idea by an older name, and `sans-serif` is the last resort every browser guarantees. This is the chain that keeps the page readable when the Google Fonts request fails.

Line 22 defines the motion curve. `cubic-bezier(.22,1,.36,1)` describes how an animation's speed varies across its duration, by giving two control points of a curve. This particular one starts fast and settles gently, the shape usually described as "ease out". Every transition in the file refers to it, so all movement on the page shares one personality.

Two tokens are defined and never used

`-accent2` (line 14, an orange) and `-shadow` (line 24) are read zero times in the entire file. Counted with `grep -c 'var(-accent2)'`, which returns `0`, and likewise for `-shadow`.
The orange is not entirely absent from the page: it appears as the literal `#ff6719` in the second avatar gradient (line 320) and `#ff9a5c` in the second proof avatar (line 580). So the colour is used and the token for it is not, which is the worst of both worlds. Either delete the token or use it.

Lines 26 to 43: the reset, and the one utility class

```
26
27  *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
28  html { color-scheme: dark; scroll-behavior: smooth; }
29  body {
30    font-family: var(--font-b);
31    background: var(--s0);
32    color: var(--ink1);
33    -webkit-font-smoothing: antialiased;
34    text-rendering: optimizeLegibility;
```

```
35   overflow-x: hidden;
36   line-height: 1.5;
37 }
38 img, video, svg { max-width: 100%; display: block; }
39 a { color: inherit; text-decoration: none; }
40 button { cursor: pointer; border: none; font-family: inherit; }
41
42 /* --- UTILITY --- */
43 .container { max-width: var(--maxw); margin: 0 auto; padding: 0 24px; }
```

Listing 24: `css/site.css`, lines 26–43

Line 27 is a reset, and it is doing three separate jobs. The selector `*` means every element; `*::before` and `*::after` extend that to the two invisible pseudo-elements CSS lets you attach to anything.

`box-sizing: border-box` is the most consequential declaration in the file. By default, if you give an element a width of 200 pixels and then 20 pixels of padding, it occupies 240 pixels: the padding is added outside the stated width. That default surprises everyone. `border-box` changes it so the stated width is the final width and padding eats into it. Every layout calculation in this file assumes it.

`margin: 0; padding: 0` removes the browser's built-in spacing on headings, paragraphs and lists, so that all spacing in the design is deliberate rather than inherited from a default that varies between browsers.

Line 28 does two things to the document root. `color-scheme: dark` tells the browser this page is dark, so that scrollbars, text-selection highlights and form controls are drawn in their dark variants rather than staying stubbornly white. `scroll-behavior: smooth` makes every jump-link glide rather than snap.

The smooth scroll is implemented twice

Line 28 already makes anchor links glide. `main.js` lines 70 to 77 then intercept every anchor link and call `scrollIntoView({behavior: 'smooth'})` to do the same thing again. Either alone is sufficient. The JavaScript version is the redundant one, and it is also the one that carries a latent bug, described in section 7.3.

Lines 29 to 37 set the page's defaults: the body typeface, the darkest surface as the background, the brightest ink as the text, and a line height of 1.5, meaning each line of text occupies one and a half times the font's size. That last number is the most important typographic decision in the file and the one most often set badly.

`-webkit-font-smoothing: antialiased` on line 33 thins text rendering on Mac displays. It is a Safari and Chrome specific property with no standard equivalent, and it is the conventional companion to light text on a dark background, which otherwise looks heavier than intended.

`overflow-x: hidden` on line 35 forbids horizontal scrolling. Here it is load-bearing rather than cosmetic: the hero tiles are positioned by percentage and some of them extend past the right edge of the window, which would otherwise let the visitor scroll sideways into empty space.

Hiding horizontal overflow treats a symptom

`overflow-x: hidden` on the body is a common fix and a blunt one: it hides the consequence of something overflowing rather than stopping the overflow. The cost is that if a genuine layout bug ever pushes content off screen, it will be invisible instead of obvious. Here the cause is known and intentional (the drifting tiles), so the trade is defensible, but it is worth recognising it as a patch.

Line 38 makes images, videos and SVGs never exceed their container's width, and makes them block-level. That second part fixes a genuinely confusing default: images are inline by default,

which reserves space beneath them for the descenders of text that is not there, producing a mysterious few pixels of gap under every image.

Line 39 strips links of their blue colour and underline, making them inherit whatever colour surrounds them. That is a deliberate design choice and an accessibility trade: colour alone now distinguishes a link from text, which fails the usual guidance. In this file every link is either a button or a footer item, so the practical harm is small.

Line 40 styles buttons, and is dead in the strict sense that this page contains no `<button>` element at all. It costs nothing and would come alive the moment one was added.

Line 43 is the only utility class in the file: a centred column no wider than 1320 pixels with 24 pixels of breathing room on each side. `margin: 0 auto` is the standard centring idiom, meaning no vertical margin and equal automatic margins left and right. Every section on the page wraps its content in this.

Lines 45 to 75: the button system

```
44
45 .btn {
46   display: inline-flex;
47   align-items: center;
48   gap: 8px;
49   padding: 13px 28px;
50   border-radius: 100px;
51   font-family: var(--font-d);
52   font-size: 15px;
53   font-weight: 600;
54   letter-spacing: -0.01em;
55   transition: transform 0.18s var(--ease), box-shadow 0.18s var(--ease),
56               background 0.18s var(--ease);
57 }
58 .btn-primary {
59   background: var(--accent);
60   color: #fff;
61 }
62 .btn-primary:hover {
63   background: var(--accentH);
64   transform: translateY(-2px);
65   box-shadow: 0 8px 24px rgba(25,103,255,.4);
66 }
67 .btn-ghost {
68   background: transparent;
69   color: var(--ink2);
70   border: 1px solid var(--border);
71 }
72 .btn-ghost:hover {
73   background: var(--s1);
74   color: var(--ink1);
75 }
```

Listing 25: `css/site.css`, lines 44–75

This is the two-class pattern the HTML uses everywhere: `.btn` holds everything both buttons share, and `.btn-primary` or `.btn-ghost` adds only what differs. Add a third button style and you write four lines, not thirty.

`display: inline-flex` on line 46 makes the button a flex container that sits in the flow of text rather than on its own line. That matters because the primary buttons contain both a word and an icon, and flex is what lines those up.

Jargon: Flexbox

A layout system that arranges an element's children in a row or a column and controls how they align and how leftover space is shared. Set `display: flex` on a parent and its children become flex items. `align-items: center` lines them up on their centres, which is how the icon in a button sits level with the text rather than on the baseline.

`gap: 8px` on line 48 puts eight pixels between the word and the icon without either of them needing a margin. Before `gap` existed this required fiddly rules targeting all-but-the-first child, and its arrival is one of the genuine improvements in modern CSS.

`border-radius: 100px` on line 50 makes a pill. Any radius larger than half the element's height produces a perfect semicircular end, so 100 is simply a number comfortably larger than needed.

Lines 55 and 56 declare the transition: three named properties, each over 0.18 seconds, each using the shared curve. Listing properties explicitly rather than writing `transition: all` is the better practice, because `all` makes the browser watch every property for changes and can animate things you never intended.

Lines 62 to 66 are the hover state. `transform: translateY(-2px)` lifts the button two pixels, and the shadow beneath it grows: together they read as the button rising towards the cursor. The shadow colour `rgba(25,103,255,.4)` is the brand blue at 40% opacity, written as red, green and blue numbers plus an alpha.

Jargon: rgba and opacity

`rgba(25,103,255,.4)` names a colour by its red, green and blue amounts from 0 to 255, plus an opacity from 0 (invisible) to 1 (solid). It is the same blue as `#1967ff`, expressed in the form that allows transparency.

Buttons have hover states and no focus states

Neither button defines `:focus` or `:focus-visible`. A visitor navigating with a keyboard rather than a mouse therefore gets whatever outline the browser draws by default, which on a dark background is often nearly invisible. Since these are the page's only interactive elements, this is the most consequential accessibility gap in the stylesheet, and it is three lines to fix.

Lines 77 to 92: the two reveal mechanisms

```
76
77 /* --- REVEAL (scroll-driven, driven by JS adding .visible) --- */
78 .reveal {
79   opacity: 0;
80   transform: translateY(28px);
81   transition: opacity 0.7s var(--ease), transform 0.7s var(--ease);
82 }
83 .reveal.visible { opacity: 1; transform: none; }
84
85 /* stagger via CSS custom property set inline */
86 .stagger > * {
87   opacity: 0;
88   transform: translateY(20px);
89   transition: opacity 0.6s var(--ease), transform 0.6s var(--ease);
90   transition-delay: calc(var(--i, 0) * 90ms);
91 }
92 .stagger.visible > * { opacity: 1; transform: none; }
```

Listing 26: `css/site.css`, lines 76–92

These sixteen lines are the entire animation contract between the stylesheet and the JavaScript, and the comment on line 77 states it: the CSS defines both states and the transition between

them, and the JavaScript's only job is to add the class `visible` at the right moment.

`.reveal` starts invisible and 28 pixels low. `.reveal.visible`, which matches an element carrying *both* classes, returns it to fully visible and its natural position. Because a transition is declared, the browser animates between the two over 0.7 seconds rather than snapping.

The stagger is the same idea with one addition. `.stagger > *` means every direct child of a staggered container, and line 90 delays each child's transition by `calc(var(-i, 0) * 90ms)`: the number the HTML wrote inline, multiplied by 90 milliseconds. Child 0 starts immediately, child 3 starts 270 milliseconds later.

Jargon: calc, and a custom property's fallback

`calc()` performs arithmetic in CSS, and can mix units. `var(-i, 0)` reads the custom property `-i` and falls back to `0` if it was never set, which is what keeps this rule safe on any element whose inline `-i` was forgotten: it simply gets no delay instead of breaking.

Why the animation lives in CSS and the trigger lives in JavaScript

This split is the reason the page degrades well. If JavaScript never runs, the class `visible` is never added, and every revealed element stays at opacity zero, invisible forever. That would be catastrophic, and it is exactly why line 767 exists: the reduced-motion block at the end of the file forces `.reveal` and `.stagger > *` back to full opacity. That block is doing double duty as an accessibility feature and as a safety net.

Lines 94 to 133: the navigation bar

```
93
94  /* --- NAV --- */
95  .nav {
96    position: fixed;
97    top: 0; left: 0; right: 0;
98    z-index: 100;
99    padding: 18px 24px;
100   display: flex;
101   align-items: center;
102   justify-content: space-between;
103   transition: background 0.3s, border-color 0.3s, box-shadow 0.3s;
104 }
105 .nav.scrolled {
106   background: rgba(5,7,13,0.88);
107   border-bottom: 1px solid var(--border);
108   box-shadow: 0 4px 24px rgba(0,0,0,.3);
109   backdrop-filter: blur(16px);
110   -webkit-backdrop-filter: blur(16px);
111 }
112 .nav-inner {
113   max-width: var(--maxw);
114   width: 100%;
115   margin: 0 auto;
116   display: flex;
117   align-items: center;
118   justify-content: space-between;
119 }
120 .nav-logo {
121   display: flex;
122   align-items: center;
123   gap: 10px;
124   font-family: var(--font-d);
125   font-size: 18px;
126   font-weight: 700;
127   letter-spacing: -0.02em;
128   color: var(--ink1);
```

```
}
.nav-logo .logo-ai {
  color: var(--accent);
}
.nav-logo svg { width: 28px; height: 28px; }
```

Listing 27: `css/site.css`, lines 93–133

`position: fixed` on line 96 pins the bar to the window rather than to the document, so it stays put while the page scrolls beneath it. Lines 97 pins it to the top and stretches it across.

`z-index: 100` on line 98 controls stacking order: when elements overlap, the higher number is drawn in front. 100 is arbitrary and generous, chosen so nothing else on the page can accidentally cover the bar.

Lines 105 to 111 are the scrolled state, the class the JavaScript adds once the page has moved more than 40 pixels. It gives the bar a background at 88% opacity, a bottom hairline, and a shadow.

`backdrop-filter: blur(16px)` on line 109 is the interesting one: it blurs whatever is *behind* the element, producing the frosted-glass effect that only works because the background is slightly transparent. Line 110 repeats it with a `-webkit-` prefix for Safari, which needed the prefixed form for years.

A doubled flex layout

`.nav` sets `display: flex` with `justify-content: space-between` (lines 100 to 102), and `.nav-inner` sets exactly the same three properties again (lines 116 to 118). The outer one has only one child, so its `space-between` has nothing to space. It is harmless and it is redundant, and it makes the layout harder to reason about than it needs to be.

Line 130 is a descendant selector: `.nav-logo .logo-ai`, with a space between the two names, means "an element with class `logo-ai` anywhere inside an element with class `nav-logo`". That space is meaningful syntax, and reading it as one thing rather than two is a common beginner error.

Lines 135 to 177: the hero shell and the floating animatior

```

/* --- HERO --- */
.hero {
  position: relative;
  min-height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  overflow: hidden;
  padding-top: 80px;
}

/* animated tile backdrop */
.hero-bg {
  position: absolute;
  inset: 0;
  pointer-events: none;
}
.tile {
  position: absolute;
  width: 176px;
  background: var(--s1);
  border: 1px solid var(--border);
  border-radius: var(--r);
  padding: 14px;
  opacity: 0;
```

```
160   animation: tileFloat 12s var(--ease) infinite;
161 }
162 .tile:nth-child(1) { top: 12%; left: 3%;   animation-delay:  0.0s; }
163 .tile:nth-child(2) { top: 18%; right: 4%;  animation-delay:  1.5s; }
164 .tile:nth-child(3) { top: 55%; left: 5%;   animation-delay:  3.0s; }
165 .tile:nth-child(4) { bottom: 18%; right: 3%; animation-delay: 4.5s; }
166 .tile:nth-child(5) { top: 42%; right: 26%; animation-delay:  6.0s; }
167 .tile:nth-child(6) { top:  8%; left: 32%;  animation-delay:  7.5s; }
168 .tile:nth-child(7) { bottom: 28%; left: 22%; animation-delay: 9.0s; }
169 .tile:nth-child(8) { top: 30%; left: 64%;  animation-delay: 10.5s; }
170
171 @keyframes tileFloat {
172   0%   { opacity: 0; transform: translateY(18px) scale(0.94); }
173   12%  { opacity: 0.8; transform: translateY(0) scale(1); }
174   72%  { opacity: 0.8; transform: translateY(-10px) scale(1); }
175   85%  { opacity: 0; transform: translateY(-18px) scale(0.94); }
176   100% { opacity: 0; transform: translateY(-18px) scale(0.94); }
177 }
```

Listing 28: `css/site.css`, lines 134–177

`position: relative` on line 137 looks like it does nothing, and it is essential. It establishes this element as the reference point for any absolutely positioned descendant. Without it, the tiles would position themselves against the whole window instead of against the hero.

Jargon: Positioning: relative and absolute

`position: absolute` takes an element out of the normal flow and places it at coordinates you give. Those coordinates are measured from the nearest ancestor that is itself positioned, which in practice means the nearest one with `position: relative`. This pairing, a relative parent and absolute children, is the standard way to place things inside a box.

`min-height: 100vh` on line 138 makes the hero at least as tall as the window. `vh` means one percent of the viewport height, so `100vh` is the full height.

`pointer-events: none` on line 149 makes the tile container transparent to the mouse: clicks pass straight through to whatever is beneath. Without it, the tiles would sit over the hero buttons and swallow clicks aimed at "Book a call". It is a one-line fix for a bug that would otherwise be maddening to diagnose, since the button would look perfectly normal and simply not respond.

Lines 152 to 161 define a tile: absolutely positioned, 176 pixels wide, using the card surface, the standard border and the standard 16-pixel radius, and starting at `opacity: 0`. It is invisible until the animation makes it visible.

Line 160 runs the animation: the named sequence `tileFloat`, over 12 seconds, using the shared curve, repeating forever.

Lines 162 to 169 place the eight tiles individually. `:nth-child(1)` matches the first child of its parent, and so on.

This is where the HTML order becomes load-bearing

Nothing here mentions `tile-blue` or `tile-orange`. The tiles are positioned purely by their *number*, so tile 5 is wherever the fifth child happens to be. Reorder the blocks in `index.html` and the layout of the hero changes, with no change to this file. That coupling is invisible from either file alone, and it is the kind of thing this document exists to make visible.

Note also that the eight delays are 1.5 seconds apart, from 0.0 up to 10.5, across a 12-second loop. That is a deliberate arrangement: eight evenly spaced entrances inside one cycle, so at any moment several tiles are visible at different stages and the backdrop never appears empty or synchronised.

Lines 171 to 177 define the animation itself.

Jargon: Keyframes

An animation defined by describing what the element looks like at particular points through its run, given as percentages of the total duration. The browser fills in every moment in between. `0%` is the start and `100%` is the end.

Read as a story: at the start the tile is invisible, 18 pixels low and slightly small. By 12% of the way through (about 1.4 seconds) it has risen into place at full size and 80% opacity. It holds there until 72%, drifting up 10 pixels. By 85% it has faded out completely while continuing to rise and shrink. The final 15% is spent invisible, which is the gap before it begins again.

Lines 179 to 218: what is inside a tile

```
178
179 .tile-thumb {
180   border-radius: 10px;
181   height: 90px;
182   margin-bottom: 10px;
183   display: flex;
184   align-items: center;
185   justify-content: center;
186   position: relative;
187   overflow: hidden;
188 }
189 .tile-thumb svg { width: 28px; height: 28px; color: rgba(255,255,255,.6); position: relative;
        z-index: 1; }
190 .tile-meta { display: flex; align-items: center; justify-content: space-between; gap: 6px; }
191 .tile-label { font-size: 11px; color: var(--ink3); font-weight: 500; }
192 .tile-metric {
193   font-family: var(--font-d);
194   font-size: 11px;
195   font-weight: 700;
196   color: var(--green);
197   background: rgba(52,211,153,.1);
198   border-radius: 20px;
199   padding: 2px 7px;
200 }
201
202 /* tile gradient variants */
203 .tile-blue .tile-thumb  { background: linear-gradient(135deg,#0a1e50,#0d2c6e); }
204 .tile-orange .tile-thumb { background: linear-gradient(135deg,#3a1800,#5a2800); }
205 .tile-teal .tile-thumb  { background: linear-gradient(135deg,#003838,#005252); }
206 .tile-violet .tile-thumb { background: linear-gradient(135deg,#1a0a40,#2a1060); }
207
208 /* fake text bars inside "post" tiles */
209 .tile-lines { padding: 0; }
210 .tile-line {
211   height: 7px;
212   border-radius: 4px;
213   background: var(--s2);
214   margin-bottom: 6px;
215 }
216 .tile-line:nth-child(1) { width: 100%; }
217 .tile-line:nth-child(2) { width: 85%; }
218 .tile-line:nth-child(3) { width: 60%; }
```

Listing 29: `css/site.css`, lines 178–218

Lines 179 to 188 shape the thumbnail area: a fixed 90-pixel-tall rounded box that centres its contents both ways using flex.

Line 189 styles the icon inside it. Note `color:` rather than `fill:` or `stroke:`: this works because the SVG in the HTML was written with `stroke="currentColor"`, so setting a text colour here reaches into the drawing. That is the pay-off for the attribute noted in section 7.1.

Line 190 lays out the caption row with `space-between`, which pushes the label to the left edge and the metric to the right, the flex idiom for a two-ended row.

Lines 192 to 200 style the metric as a small green pill: green text on a green background at 10% opacity, which is the standard recipe for a tinted badge that suits any surface beneath it.

Lines 203 to 206 are the four colour variants. Each is a descendant selector: the gradient applies to a `.tile-thumb` *inside* a `.tile-blue`, not to the tile itself. `linear-gradient(135deg, A, B)` blends from colour A to colour B along a line at 135 degrees, which runs from top-left to bottom-right.

Line 209 is the dead rule identified in section 7.1: both places that use `.tile-lines` override its padding inline, so this never applies.

Lines 210 to 218 draw the fake text bars: 7 pixels tall, rounded, in the `s2` surface, with the three widths that make them read as a paragraph. Note that tile 8 overrides the third width inline to 50%, so these are defaults rather than rules.

Lines 220 to 243: the statistic tiles and the readability overlay

```
219
220 /* stats tile */
221 .tile-stat-num {
222   font-family: var(--font-d);
223   font-size: 26px;
224   font-weight: 700;
225   color: var(--ink1);
226   line-height: 1;
227   margin-bottom: 4px;
228 }
229 .tile-stat-lbl { font-size: 11px; color: var(--ink3); }
230
231 /* overlay - keeps center readable */
232 .hero-overlay {
233   position: absolute;
234   inset: 0;
235   background:
236     radial-gradient(ellipse 55% 65% at 50% 50%,
237       rgba(5,7,13,.9) 0%,
238       rgba(5,7,13,.55) 70%,
239       transparent 100%),
240     linear-gradient(to bottom,
241       rgba(5,7,13,.2) 0%,
242       rgba(5,7,13,.85) 100%);
243 }
```

Listing 30: `css/site.css`, lines 219–243

`line-height: 1` on line 226 is deliberate. The page-wide default of 1.5 is right for paragraphs and wrong for a large number, where it leaves a visible gap above and below. Setting it to 1 makes the line exactly as tall as the digits.

Lines 231 to 243 are the overlay, and they are the most sophisticated piece of styling in the file. Two gradients are stacked in one declaration, separated by a comma, and in CSS the *first* one listed is drawn on top.

The first is a radial gradient: an ellipse 55% wide and 65% tall, centred, going from 90% opaque near-black at its centre, through 55% at seven-tenths of the way out, to fully transparent at the edge. That is a dark cloud over the middle of the hero, exactly where the headline sits.

The second is a linear gradient running down the whole area, from 20% opaque at the top to 85% at the bottom, which darkens the lower half so the hero blends into the section beneath it.

Together they solve the real problem with a busy animated backdrop: text over moving content is hard to read, and the contrast changes as the content moves. Rather than dimming the tiles themselves, the design puts a shaped veil between them and the text.

Jargon: inset

`inset: 0` is shorthand for `top: 0; right: 0; bottom: 0; left: 0`, which on an absolutely positioned element means "stretch to fill the parent completely".

Lines 245 to 324: the hero's content

```
244
245 .hero-content {
246   position: relative;
247   z-index: 2;
248   text-align: center;
249   max-width: 700px;
250   padding: 0 24px;
251 }
252 .hero-badge {
253   display: inline-flex;
254   align-items: center;
255   gap: 8px;
256   background: rgba(25,103,255,.12);
257   border: 1px solid rgba(25,103,255,.3);
258   border-radius: 100px;
259   padding: 6px 14px;
260   font-size: 13px;
261   font-weight: 600;
262   color: var(--accentH);
263   letter-spacing: 0.01em;
264   margin-bottom: 28px;
265 }
266 .hero-badge-dot {
267   width: 7px; height: 7px;
268   border-radius: 50%;
269   background: var(--accent);
270   animation: pulse 2s ease-in-out infinite;
271 }
272 @keyframes pulse {
273   0%, 100% { opacity: 1; transform: scale(1); }
274   50%      { opacity: 0.5; transform: scale(0.7); }
275 }
```

Listing 31: `css/site.css`, lines 244–275

`z-index: 2` on line 247 lifts the content above the tiles and the overlay, which is what makes the headline readable rather than buried. Note that `z-index` only works on positioned elements, which is why line 246 sets `position: relative` even though nothing is being moved.

The badge (lines 252 to 265) uses the tinted-pill recipe again: brand blue at 12% for the fill and 30% for the border, with the brighter hover blue for the text so it stays legible against its own tint.

Lines 266 to 275 define the pulsing dot: a 7-pixel square made circular with a 50% radius, running a two-second pulse forever. The keyframes group `0%, 100%` on one line, which is the idiom for an animation that returns to where it started: full size at both ends, shrunk to 70% and half-faded in the middle.

```
276 .hero-h1 {
277   font-family: var(--font-d);
278   font-size: clamp(42px, 6.5vw, 80px);
279   font-weight: 700;
280   letter-spacing: -0.03em;
281   line-height: 1.06;
282   color: var(--ink1);
283   margin-bottom: 36px;
284 }
285 .hero-h1 em {
```

```
286   font-style: normal;
287   color: var(--accent);
288 }
289 .hero-actions {
290   display: flex;
291   align-items: center;
292   justify-content: center;
293   gap: 14px;
294   flex-wrap: wrap;
295   margin-bottom: 48px;
296 }
297 .hero-social-proof {
298   display: flex;
299   align-items: center;
300   gap: 12px;
301   justify-content: center;
302 }
303 .avatars { display: flex; }
304 .avatar {
305   width: 34px; height: 34px;
306   border-radius: 50%;
307   border: 2px solid var(--s0);
308   margin-left: -10px;
309   display: flex;
310   align-items: center;
311   justify-content: center;
312   font-family: var(--font-d);
313   font-size: 12px;
314   font-weight: 700;
315   color: #fff;
316   flex-shrink: 0;
317 }
318 .avatars .avatar:first-child { margin-left: 0; }
319 .av1 { background: linear-gradient(135deg,#1967ff,#3f82ff); }
320 .av2 { background: linear-gradient(135deg,#ff6719,#ff8c4a); }
321 .av3 { background: linear-gradient(135deg,#34d399,#10b981); }
322 .av4 { background: linear-gradient(135deg,#a78bfa,#7c3aed); }
323 .hero-proof-text { font-size: 13px; color: var(--ink3); }
324 .hero-proof-text strong { color: var(--ink2); }
```

Listing 32: `css/site.css`, lines 276–324

Line 278 is the most useful single line in the file to learn from.

Jargon: clamp

`clamp(minimum, preferred, maximum)` picks the preferred value, but never lets it fall below the minimum or rise above the maximum. Here the preferred value is `6.5vw`, meaning 6.5% of the window width, so the headline scales continuously with the window between a floor of 42 pixels and a ceiling of 80. One line replaces what used to need several media queries.

Line 280 sets `letter-spacing: -0.03em`, pulling the letters slightly closer together. Large display type looks loose at its default spacing, and negative tracking at large sizes is standard practice. The unit `em` means "relative to this element's font size", so the tightening scales with the text.

Lines 285 to 288 are the rule flagged in section 7.1: it takes the `<em>` tag, removes the italics it normally implies, and colours it blue instead.

Line 294 sets `flex-wrap: wrap` on the button row, so on a narrow screen the second button drops onto its own line instead of overflowing. That single property is why the hero needs no special mobile rule for its buttons.

Lines 303 to 318 build the overlapping avatar row, and the technique is worth naming: each

circle has a negative left margin of 10 pixels, pulling it over its neighbour, and a 2-pixel border in the page background colour, which reads as a gap punched out of the circle behind it. Line 318 then cancels the negative margin on the first one so the row does not start shifted left.

`flex-shrink: 0` on line 316 stops the circles being squashed when space is tight. Flex items shrink by default, and circles that shrink become ovals.

Four literal colours where tokens exist

Lines 319 to 322 write `#1967ff`, `#3f82ff`, `#ff6719` and `#34d399` directly. Three of those four have tokens (`-accent`, `-accentH`, `-green`) and the fourth has one that is otherwise unused (`-accent2`). Gradients accept custom properties perfectly well, so there is no technical reason for the literals. This is the same inconsistency as the inline styles in the HTML, and together they mean the token system governs most of the page rather than all of it.

Lines 326 to 359: the metrics band

```
325
326 /* --- METRICS BAND --- */
327 .metrics {
328   padding: 80px 0;
329   border-top: 1px solid var(--borderS);
330   border-bottom: 1px solid var(--borderS);
331 }
332 .metrics-grid {
333   display: grid;
334   grid-template-columns: repeat(4, 1fr);
335   gap: 2px;
336 }
337 .metric-item {
338   padding: 36px 24px;
339   text-align: center;
340   border-right: 1px solid var(--borderS);
341 }
342 .metric-item:last-child { border-right: none; }
343 .metric-num {
344   font-family: var(--font-d);
345   font-size: clamp(36px, 4.5vw, 60px);
346   font-weight: 700;
347   letter-spacing: -0.03em;
348   line-height: 1;
349   color: var(--ink1);
350   margin-bottom: 8px;
351 }
352 .metric-num .accent { color: var(--accent); }
353 .metric-label {
354   font-size: 13px;
355   color: var(--ink3);
356   font-weight: 500;
357   text-transform: uppercase;
358   letter-spacing: 0.06em;
359 }
```

Listing 33: `css/site.css`, lines 325–359

Line 333 introduces the second layout system in the file.

Jargon: CSS Grid

A layout system for two dimensions, as opposed to flexbox's one. `grid-template-columns: repeat(4, 1fr)` means four columns, each taking one equal share of the space. `1fr` is a fraction of what is left over after any fixed sizes are accounted for.

The separators here are drawn as a right border on every item (line 340), with the last one removed (line 342). That is a different technique from the one used later by the services grid, which draws its lines using gaps. Both work; using both in one file is an inconsistency, and the border approach is the one that needed a special case.

Line 352, `.metric-num .accent`, styles an element with class `accent` inside a metric number. **No such element exists** anywhere in `index.html`. This rule is dead.

Lines 353 to 359 style the label beneath each number: small, dim, uppercase, with positive letter-spacing. Note the tracking is positive here and negative on the big numbers above. That is not an inconsistency but a rule of typography: large text needs tightening, small uppercase text needs opening up or it becomes a smudge.

Lines 361 to 379: shared section headings

```
360
361 /* --- PIPELINE --- */
362 .pipeline-section { padding: 120px 0; }
363 .section-kicker {
364   font-size: 12px;
365   font-weight: 700;
366   text-transform: uppercase;
367   letter-spacing: 0.1em;
368   color: var(--accent);
369   margin-bottom: 14px;
370 }
371 .section-title {
372   font-family: var(--font-d);
373   font-size: clamp(30px, 4vw, 52px);
374   font-weight: 700;
375   letter-spacing: -0.025em;
376   line-height: 1.1;
377   color: var(--ink1);
378   margin-bottom: 72px;
379 }
```

Listing 34: `css/site.css`, lines 360–379

Although these sit under the pipeline banner, `.section-kicker` and `.section-title` are used by all four remaining sections. Their placement here is a small organisational untidiness: a reader looking for the heading styles would reasonably expect them near the top with the other shared pieces, not inside the pipeline block.

The kicker is the small blue uppercase line (“How it works”, “The work”) and the title is the large heading beneath it. Both follow the same tracking rule as the metrics band.

The kicker’s contrast is worth measuring

`-accent` is `#1967ff`, a saturated blue, and the kicker sets it at 12 pixels in uppercase on the near-black page background. Small text needs more contrast than large text, not less. This document does not have a measured contrast ratio for that exact pairing and so does not assert a failure, but it is the first thing to check in an accessibility audit, along with the 11-pixel service tags which use the dimmest ink on the same background.

Lines 381 to 450: the pipeline

```
380
381 .pipeline {
382   display: flex;
383   align-items: center;
384   gap: 0;
385   position: relative;
386   overflow: hidden;
387 }
```

```
388 .pipeline-node {
389   flex: 1;
390   display: flex;
391   flex-direction: column;
392   align-items: center;
393   gap: 16px;
394   position: relative;
395   z-index: 1;
396   opacity: 0;
397   transform: scale(0.85);
398   transition: opacity 0.5s var(--ease), transform 0.5s var(--ease);
399 }
400 .pipeline.visible .pipeline-node:nth-child(1) { opacity: 1; transform: none; transition-delay:
        0.0s; }
401 .pipeline.visible .pipeline-node:nth-child(3) { opacity: 1; transform: none; transition-delay:
        0.2s; }
402 .pipeline.visible .pipeline-node:nth-child(5) { opacity: 1; transform: none; transition-delay:
        0.4s; }
403 .pipeline.visible .pipeline-node:nth-child(7) { opacity: 1; transform: none; transition-delay:
        0.6s; }
404 .pipeline.visible .pipeline-node:nth-child(9) { opacity: 1; transform: none; transition-delay:
        0.8s; }
405
406 .pipeline-connector {
407   flex: 0 0 auto;
408   width: 60px;
409   height: 2px;
410   background: var(--border);
411   position: relative;
412   overflow: hidden;
413 }
414 .pipeline-connector::after {
415   content: '';
416   position: absolute;
417   top: 0; left: 0;
418   height: 100%;
419   width: 0;
420   background: linear-gradient(to right, var(--accent), var(--accentH));
421   transition: width 0.6s var(--ease);
422 }
423 .pipeline.visible .pipeline-connector:nth-child(2)::after { width: 100%; transition-delay:
        0.15s; }
424 .pipeline.visible .pipeline-connector:nth-child(4)::after { width: 100%; transition-delay:
        0.35s; }
425 .pipeline.visible .pipeline-connector:nth-child(6)::after { width: 100%; transition-delay:
        0.55s; }
426 .pipeline.visible .pipeline-connector:nth-child(8)::after { width: 100%; transition-delay:
        0.75s; }
```

Listing 35: `css/site.css`, lines 380–426

This is the most intricate animation in the file and it is worth reading closely, because it is doing something the stagger mechanism could not.

The pipeline is a flex row of nine children alternating node, connector, node, connector and so on: five nodes at odd positions and four connectors at even ones. That alternation is why the selectors count 1, 3, 5, 7, 9 for nodes and 2, 4, 6, 8 for connectors.

`flex: 1` on the nodes (line 389) tells each to take an equal share of the leftover space. `flex: 0 0 auto` on the connectors (line 407) tells them the opposite: do not grow, do not shrink, stay exactly 60 pixels. So the nodes spread and the connectors stay fixed.

Each node begins invisible and at 85% size, and the `.visible` rules bring it to full size with an increasing delay, 0.2 seconds apart. The connectors do something different: each has an `::after` pseudo-element, an invisible extra box CSS creates on demand, absolutely positioned, starting at

zero width and filled with a blue gradient. When the section becomes visible its width transitions to 100%, so the blue appears to flow along the line.

Jargon: Pseudo-element

An extra box that CSS creates inside an element without any corresponding tag in the HTML, written as `::before` or `::after`. It requires the `content` property to exist at all, which is why line 415 sets `content: ''`: an empty string is enough to bring it into being. It is the standard way to draw a decorative shape without polluting the markup.

The delays interleave deliberately: node 1 at 0.0, connector at 0.15, node 3 at 0.2, connector at 0.35, and so on. The line starts drawing before the next node lands, which is what makes the sequence read as flowing rather than as a series of separate appearances.

This animation cannot survive an edit to the HTML

Every selector here depends on the exact child positions in `index.html`. Add a sixth step, or wrap the nodes in a container, and every one of these nine rules silently stops matching: the pipeline would simply never appear, staying at opacity zero forever, with no error anywhere. Selecting by class (`.pipeline-node:nth-of-type(n)`) would have been sturdier for the same effect.

```
427
428 .pipeline-icon {
429   width: 72px; height: 72px;
430   border-radius: 20px;
431   background: var(--s1);
432   border: 1px solid var(--border);
433   display: flex;
434   align-items: center;
435   justify-content: center;
436   transition: border-color 0.3s, box-shadow 0.3s;
437 }
438 .pipeline-icon svg { width: 30px; height: 30px; color: var(--accent); }
439 .pipeline-node:hover .pipeline-icon {
440   border-color: var(--accent);
441   box-shadow: 0 0 24px rgba(25,103,255,.25);
442 }
443 .pipeline-lbl {
444   font-family: var(--font-d);
445   font-size: 13px;
446   font-weight: 600;
447   color: var(--ink2);
448   text-align: center;
449   white-space: nowrap;
450 }
```

Listing 36: `css/site.css`, lines 427–450

Lines 439 to 442 give each icon box a blue border and a soft blue glow when the mouse is over its node. A shadow with no offset and a large blur, as here, reads as a glow rather than a shadow.

`white-space: nowrap` on line 449 stops labels such as "Auto-Post" wrapping onto two lines and disturbing the row's alignment. It is overridden at line 739 for narrow screens, where the pipeline stacks vertically and there is no alignment left to protect.

Lines 452 to 539: the portfolio cards

```
451
452 /* --- PORTFOLIO --- */
453 .portfolio-section { padding: 0 0 120px; }
454 .portfolio-grid {
```

```
455   display: grid;
456   grid-template-columns: repeat(3, 1fr);
457   gap: 16px;
458 }
459 .port-card {
460   background: var(--s1);
461   border: 1px solid var(--border);
462   border-radius: var(--r);
463   overflow: hidden;
464   transition: transform 0.22s var(--ease), border-color 0.22s, box-shadow 0.22s;
465   cursor: pointer;
466 }
467 .port-card:hover {
468   transform: translateY(-4px);
469   border-color: rgba(25,103,255,.4);
470   box-shadow: 0 16px 48px rgba(0,0,0,.4);
471 }
472 .port-thumb {
473   aspect-ratio: 16/9;
474   position: relative;
475   display: flex;
476   align-items: center;
477   justify-content: center;
478   overflow: hidden;
479 }
480 .port-thumb-overlay {
481   position: absolute;
482   inset: 0;
483   background: rgba(5,7,13,.25);
484   display: flex;
485   align-items: center;
486   justify-content: center;
487   opacity: 0;
488   transition: opacity 0.22s;
489 }
490 .port-card:hover .port-thumb-overlay { opacity: 1; }
491 .port-play {
492   width: 52px; height: 52px;
493   border-radius: 50%;
494   background: rgba(255,255,255,.15);
495   backdrop-filter: blur(8px);
496   display: flex;
497   align-items: center;
498   justify-content: center;
499   border: 1.5px solid rgba(255,255,255,.3);
500 }
501 .port-play svg { width: 22px; height: 22px; color: #fff; margin-left: 2px; }
502
503 /* gradient thumbnails - replace with real video posters */
504 .pt1 { background: linear-gradient(135deg,#0a1840 0%,#0d2c6e 50%,#0a1840 100%); }
505 .pt2 { background: linear-gradient(135deg,#2a0e00 0%,#5a2800 50%,#2a0e00 100%); }
506 .pt3 { background: linear-gradient(135deg,#001a28 0%,#003855 50%,#001a28 100%); }
507 .pt4 { background: linear-gradient(135deg,#150050 0%,#2d0080 50%,#150050 100%); }
508 .pt5 { background: linear-gradient(135deg,#0a2800 0%,#1a4800 50%,#0a2800 100%); }
509 .pt6 { background: linear-gradient(135deg,#280014 0%,#500028 50%,#280014 100%); }
```

Listing 37: `css/site.css`, lines 451–509

`overflow: hidden` on line 463 is what makes the rounded corners work: without it, the gradient thumbnail inside would square off the top two corners it is supposed to follow.

`cursor: pointer` on line 465 changes the mouse to a pointing hand, which conventionally means clickable. As section 7.1 noted, the card is not clickable, so this is a promise the page does not keep.

`aspect-ratio: 16/9` on line 473 reserves a widescreen-shaped area regardless of width. This is worth appreciating: it means the layout does not shift when a real image eventually loads, because the space is already the right shape. It is the modern replacement for a well-known hack involving percentage padding.

The overlay (lines 480 to 489) sits over the whole thumbnail at zero opacity and fades to one on hover, revealing the play button beneath a 25% black wash. The play button itself (lines 491 to 500) is a frosted circle: white at 15% opacity with an 8-pixel backdrop blur and a brighter rim.

Line 501 gives the play triangle `margin-left: 2px`. That is not an accident. A triangle's visual centre of mass sits left of its geometric centre, so a perfectly centred play icon looks slightly left. Nudging it two pixels right is a standard correction, and noticing it is a good sign that whoever built this cared about detail.

Lines 504 to 509 are the six placeholder gradients, each a three-stop blend that returns to its starting colour, so it reads as a soft sheen rather than a slope. The comment on line 503 says outright that they are standing in for real video posters that never arrived.

```
510
511 .port-badge {
512   position: absolute;
513   top: 10px; right: 10px;
514   background: rgba(5,7,13,.75);
515   backdrop-filter: blur(8px);
516   border: 1px solid var(--border);
517   border-radius: 20px;
518   padding: 4px 10px;
519   font-size: 11px;
520   font-weight: 600;
521   color: var(--green);
522 }
523 .port-info { padding: 16px; }
524 .port-platform {
525   display: flex;
526   align-items: center;
527   gap: 6px;
528   font-size: 11px;
529   color: var(--ink3);
530   margin-bottom: 6px;
531 }
532 .port-platform svg { width: 13px; height: 13px; }
533 .port-title {
534   font-family: var(--font-d);
535   font-size: 15px;
536   font-weight: 600;
537   color: var(--ink1);
538   letter-spacing: -0.01em;
539 }
```

Listing 38: `css/site.css`, lines 510–539

The badge is pinned to the thumbnail's top-right corner, which works because `.port-thumb` set `position: relative` on line 474.

The card title on lines 533 to 539 is the only place in the file where the display typeface is used at a small size (15 pixels). Everywhere else Space Grotesk appears it is either large (a heading, a metric) or tiny and uppercase (a label), so this is the one instance of it doing ordinary work.

Lines 541 to 604: the client proof cards

```
540
541 /* --- CLIENT PROOF --- */
542 .proof-section { padding: 0 0 120px; }
543 .proof-grid {
544   display: grid;
```

```
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}
.proof-card {
  background: var(--s1);
  border: 1px solid var(--border);
  border-radius: var(--r);
  padding: 28px;
  display: flex;
  flex-direction: column;
  gap: 20px;
  transition: border-color 0.22s, transform 0.22s var(--ease);
}
.proof-card:hover {
  border-color: rgba(25,103,255,.3);
  transform: translateY(-3px);
}
.proof-header {
  display: flex;
  align-items: center;
  gap: 14px;
}
.proof-avatar {
  width: 52px; height: 52px;
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
  font-family: var(--font-d);
  font-size: 20px;
  font-weight: 700;
  color: #fff;
  flex-shrink: 0;
}
.pa1 { background: linear-gradient(135deg,#1967ff,#3f82ff); }
.pa2 { background: linear-gradient(135deg,#ff6719,#ff9a5c); }
.pa3 { background: linear-gradient(135deg,#7c3aed,#a78bfa); }

.proof-name {
  font-family: var(--font-d);
  font-size: 16px;
  font-weight: 700;
  color: var(--ink1);
  letter-spacing: -0.01em;
}
.proof-company { font-size: 13px; color: var(--ink3); margin-top: 2px; }
.proof-metric {
  display: flex;
  align-items: baseline;
  gap: 6px;
}
.proof-num {
  font-family: var(--font-d);
  font-size: 36px;
  font-weight: 700;
  color: var(--accent);
  letter-spacing: -0.03em;
  line-height: 1;
}
.proof-unit { font-size: 14px; color: var(--ink3); }
```

Listing 39: `css/site.css`, lines 540–604

The proof card is a vertical flex column with a 20-pixel gap, so its two halves (the header and

the metric) space themselves without margins.

`align-items: baseline` on line 594 is the detail worth learning. The metric row holds a 36-pixel number beside 14-pixel text. Centring them would look wrong, because the eye reads the two as sitting on a shared line of writing. `baseline` aligns them on that invisible line the letters sit on, which is what a typesetter would do.

The three avatar gradients (lines 579 to 581) repeat the same literal-colour issue as the hero avatars, and `pa3` introduces two purples that exist nowhere in the token block at all.

Lines 606 to 654: the services grid

```
605
606 /* --- SERVICES --- */
607 .services-section { padding: 0 0 120px; }
608 .services-grid {
609   display: grid;
610   grid-template-columns: repeat(3, 1fr);
611   gap: 2px;
612   background: var(--border);
613   border-radius: var(--r);
614   overflow: hidden;
615 }
616 .service-item {
617   background: var(--s0);
618   padding: 40px 32px;
619   display: flex;
620   flex-direction: column;
621   gap: 16px;
622   transition: background 0.22s;
623 }
624 .service-item:hover { background: var(--s1); }
625 .service-icon {
626   width: 52px; height: 52px;
627   border-radius: var(--rsm);
628   background: rgba(25,103,255,.1);
629   border: 1px solid rgba(25,103,255,.2);
630   display: flex;
631   align-items: center;
632   justify-content: center;
633   transition: background 0.22s, border-color 0.22s;
634 }
635 .service-icon svg { width: 22px; height: 22px; color: var(--accentH); }
636 .service-item:hover .service-icon {
637   background: rgba(25,103,255,.18);
638   border-color: rgba(25,103,255,.45);
639 }
640 .service-name {
641   font-family: var(--font-d);
642   font-size: 17px;
643   font-weight: 700;
644   color: var(--ink1);
645   letter-spacing: -0.01em;
646 }
647 .service-tag {
648   display: inline-block;
649   font-size: 11px;
650   font-weight: 600;
651   color: var(--ink3);
652   text-transform: uppercase;
653   letter-spacing: 0.07em;
654 }
```

Listing 40: `css/site.css`, lines 605–654

Lines 608 to 615 use the technique mentioned in section 7.1 and it deserves its own explanation, because it is genuinely clever and genuinely fragile.

Hairlines made of gaps

The grid is given a 2-pixel gap and a background of the border colour. Each cell is then given the page background. Because the cells cover the grid's background everywhere except in the gaps, the gaps show through as 2-pixel lines. The result is a perfect internal grid of hairlines with no border rules and no special case for the last item in a row, which is exactly what the metrics band above needed (`:last-child { border-right: none }`).
The fragility is that it depends on the cell background matching the page. Put this grid on a different surface without changing `.service-item`, and the cells become visible rectangles of the wrong colour.

`border-radius` plus `overflow: hidden` on lines 613 and 614 round the whole grid and clip the corner cells to match.

Line 647 sets `display: inline-block` on the tag. Since the tag is inside a flex column, it would already behave as a block, so this has no visible effect: it is a harmless leftover.

Lines 656 to 725: the booking panel and the footer

```
655
656 /* --- BOOK A CALL --- */
657 .book-section { padding: 0 0 140px; }
658 .book-wrap {
659   background: var(--s1);
660   border: 1px solid var(--border);
661   border-radius: 24px;
662   padding: 72px 48px;
663   text-align: center;
664   position: relative;
665   overflow: hidden;
666 }
667 .book-wrap::before {
668   content: '';
669   position: absolute;
670   top: -100px; left: 50%;
671   transform: translateX(-50%);
672   width: 600px; height: 300px;
673   background: radial-gradient(ellipse, rgba(25,103,255,.12) 0%, transparent 70%);
674   pointer-events: none;
675 }
676 .book-eyebrow {
677   font-size: 12px;
678   font-weight: 700;
679   text-transform: uppercase;
680   letter-spacing: 0.1em;
681   color: var(--accent);
682   margin-bottom: 16px;
683 }
684 .book-title {
685   font-family: var(--font-d);
686   font-size: clamp(28px, 4vw, 48px);
687   font-weight: 700;
688   letter-spacing: -0.025em;
689   color: var(--ink1);
690   margin-bottom: 12px;
691 }
692 .book-sub {
693   font-size: 16px;
694   color: var(--ink3);
695   margin-bottom: 36px;
```

```
}
.book-cta-row {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 14px;
  flex-wrap: wrap;
}
.book-detail {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 6px;
  font-size: 13px;
  color: var(--ink3);
  margin-top: 20px;
}
.book-detail svg { width: 15px; height: 15px; }
.book-detail span { color: var(--ink2); }

/* --- FOOTER --- */
footer {
  border-top: 1px solid var(--borderS);
  padding: 32px 24px;
  text-align: center;
  font-size: 13px;
  color: var(--ink3);
}
footer a { color: var(--ink3); transition: color 0.15s; }
footer a:hover { color: var(--ink1); }
```

Listing 41: `css/site.css`, lines 655–725

Lines 667 to 675 add a blue glow above the booking panel using an `::before` pseudo-element: a 600 by 300 ellipse of brand blue at 12% opacity, positioned 100 pixels above the panel's top edge and centred with the standard `left: 50%` plus `translateX(-50%)` idiom. Because the panel has `overflow: hidden`, only the bottom part of the glow is visible, which is what makes it read as light spilling in from above rather than as a floating oval.

`pointer-events: none` on line 674 appears for the second time in the file, for the same reason as before: this decorative layer sits over the panel and must not intercept the click aimed at the booking button.

The footer is deliberately unremarkable: one hairline, centred dim text, and links that brighten on hover over 0.15 seconds.

Lines 727 to 768: the responsive rules

```

/* --- RESPONSIVE --- */
@media (max-width: 900px) {
  .metrics-grid { grid-template-columns: repeat(2, 1fr); }
  .metric-item { border-right: none; border-bottom: 1px solid var(--borderS); }
  .metric-item:nth-child(odd) { border-right: 1px solid var(--borderS); }
  .metric-item:nth-child(3),
  .metric-item:nth-child(4) { border-bottom: none; }

  .pipeline { flex-direction: column; }
  .pipeline-connector { width: 2px; height: 40px; }
  .pipeline-connector::after { width: 100% !important; height: 0; }
  .pipeline.visible .pipeline-connector::after { height: 100%; width: 100% !important; }
  .pipeline-lbl { white-space: normal; }

  .portfolio-grid { grid-template-columns: repeat(2, 1fr); }
  .proof-grid     { grid-template-columns: 1fr; }
```

```
743   .services-grid  { grid-template-columns: repeat(2, 1fr); }
744
745   .tile { display: none; }
746   .tile:nth-child(1),
747   .tile:nth-child(2),
748   .tile:nth-child(3) { display: block; }
749   .tile:nth-child(1) { top: 10%; left: 2%; width: 140px; }
750   .tile:nth-child(2) { top: 15%; right: 2%; width: 140px; }
751   .tile:nth-child(3) { display: none; }
752 }
753
754 @media (max-width: 600px) {
755   .portfolio-grid { grid-template-columns: 1fr; }
756   .services-grid  { grid-template-columns: 1fr; }
757   .book-wrap { padding: 48px 24px; }
758   .hero-h1 { font-size: 38px; }
759   .tile { display: none; }
760 }
761
762 @media (prefers-reduced-motion: reduce) {
763   *, *::before, *::after {
764     animation-duration: .001ms !important;
765     transition-duration: .001ms !important;
766   }
767   .reveal, .stagger > * { opacity: 1; transform: none; }
768 }
```

Listing 42: `css/site.css`, lines 726–768

Jargon: Media query

A block of CSS that only applies when a condition about the display is true. `@media (max-width: 900px)` means "apply everything in here only when the window is 900 pixels wide or narrower". Because it comes later in the file, its rules override the equivalents above.

The 900-pixel block does five things. The metrics band folds from four columns to two, and its dividing lines are rebuilt for the new shape: the right border is removed from everything, a bottom border added to everything, the right border put back on odd-numbered items so there is still a line down the middle, and the bottom border removed from items 3 and 4 so the block does not end in a stray line. That is four rules to reposition two lines, and it is the price of the border technique that the services grid avoided.

The pipeline turns vertical (line 735) and its connectors become 2 pixels wide and 40 tall (line 736). Lines 737 and 738 then rebuild the fill animation to run downwards instead of across, growing height rather than width, and use `!important` to force the width value past the more specific rules further up the file.

Jargon: !important

A marker that forces a declaration to win against the normal precedence rules. It is generally considered a last resort, because it breaks the predictability that makes CSS manageable. It appears exactly twice in this file, both on line 737 and 738, both to override the horizontal fill animation.

The three content grids each drop a column, and the hero tiles are reduced from eight to two.

Three lines of dead responsive code

Line 745 hides every tile. Lines 746 to 748 then show tiles 1, 2 and 3. Line 751 hides tile 3 again. Because line 751 comes later and has identical specificity, it wins, so tile 3 is hidden and the `display: block` it was given three lines earlier does nothing at all.
The author evidently decided three tiles were too many for a tablet and removed the third by adding a rule instead of editing the existing one. It is harmless, and it is exactly the sort of thing that makes a reader doubt their understanding of the cascade. It should be a two-line rule listing tiles 1 and 2.

The 600-pixel block hides the tiles entirely, collapses the last two grids to one column, tightens the booking panel's padding, and fixes the headline at 38 pixels, overriding the `clamp` whose floor was 42.

The final block, lines 762 to 768, is the accessibility one, and it is the best thing in the file.

What `prefers-reduced-motion` is and why this block matters

Operating systems let people ask for reduced motion, usually because animation causes them nausea, dizziness or migraine. The browser passes that preference to the page, and this block responds to it.
It does two things. It crushes every animation and transition on the page to a thousandth of a millisecond, which is effectively instant rather than disabled, a deliberate choice because setting them to zero can stop some browsers firing the events that mark an animation as finished. Then, crucially, line 767 forces every revealed and staggered element to full opacity, because those elements start invisible and would otherwise never appear once their transitions were neutered.
That second line is what turns a well-meaning block into a correct one, and it is the detail most implementations of this pattern get wrong.

The reduced-motion block cannot reach the counting numbers

This block governs CSS only. The four metric numbers are animated by JavaScript, counting up over 2.2 seconds, and `main.js` never checks this preference. A visitor who has asked their system for reduced motion still gets four numbers whirring upward.
The fix is four lines in `main.js`, given in section 7.3. This is the single clearest defect in the project: not because it is severe, but because the intent to handle it correctly is demonstrably there in the CSS and simply was not carried across to the other file.

What it connects to

Named by: `new-design-reference/index.html`, line 16, and nothing else.

Names: no other file. It refers to no images, no fonts by file (it names font *families*, which the HTML is responsible for downloading), and imports nothing.

Coupled to `index.html` in three ways that no tool checks: by class name throughout, by child position for the hero tiles (lines 162 to 169) and the pipeline (lines 400 to 426), and by the shared contract that JavaScript adds the class `visible`.

Coupled to `js/main.js` through exactly two class names, `scrolled` and `visible`, both of which the JavaScript adds and this file defines.

Concepts introduced

This file is where most of the page's technical vocabulary first appears: custom properties, the cascade, flexbox, grid, pseudo-elements, pseudo-classes, keyframe animations, transitions, media

queries, `clamp`, `calc`, `aspect-ratio`, `backdrop-filter`, stacking with `z-index`, and `prefers-reduced-motion`. Each is defined at its first use above and collected in the glossary.

How it breaks

- **Delete this file and the page still works**, in the sense that every word remains readable. It simply looks like an unstyled document, and the hero tiles appear as a plain stacked list.
- **Rename a class here without renaming it in the HTML** and that element silently loses its styling. Nothing reports it.
- **Add or remove a hero tile or a pipeline step** and the position rules stop lining up, silently.
- **Remove `pointer-events: none` on line 149** and the hero tiles begin swallowing clicks meant for the buttons beneath them.
- **Remove `position: relative` on line 137** and all eight tiles jump to positions measured against the whole window instead of the hero.
- **A missing semicolon** makes the browser discard the rest of that declaration block. Unlike a program, CSS never reports an error: it drops what it cannot parse and carries on, which is why a stylesheet bug shows up as something looking wrong rather than as a message.

Honest findings for this file

Two unused tokens (lines 14 and 24); one dead rule with no matching element (line 352); one rule permanently overridden by inline styles (line 209); three lines of dead responsive code (lines 746 to 748 versus 751); a doubled flex layout in the navigation (lines 100 to 102 and 116 to 118); smooth scrolling implemented twice, here and in JavaScript; literal colours used where tokens exist (lines 319 to 322, 579 to 581); two different techniques for drawing dividing lines in one file; no focus states on either button; and a reduced-motion block that is exemplary within CSS and cannot reach the one animation that lives outside it.

7.3 `new-design-reference/js/main.js`

JavaScript, 79 lines. Everything on the page that reacts. It does three things and nothing else: it makes the navigation bar grow a frosted background once you scroll, it counts the four metric numbers up from zero when they come into view, and it fades most blocks upward as they arrive.

Where this came from: new-design-reference/js/main.js

Hand-written, and its own header comment says as much on line 3: `Zero external dependencies.` That claim is true and checkable: the file imports nothing, requires nothing, and the repository has no `package.json` that could supply a library.
It arrived complete in the first commit, `0d77d51`, and has never been edited. The patterns in it (an immediately invoked function wrapper, an `IntersectionObserver` factory, a cubic ease-out counter) are the standard vocabulary of vanilla JavaScript rather than the fingerprint of any particular source, so no upstream is claimed. **Confidence: certain for hand-written and dependency-free; no provenance evidence found for any specific inspiration, and none is invented here.**

Why it exists

Without this file the page is complete, readable and static. Two of its three jobs are pure decoration and would not be missed. The third is not: the metric numbers are written into the HTML as the character `0`, with their real values hidden in data attributes, so with this file gone the page permanently advertises zero projects delivered and zero views.

The single most important line in this file is in another file

`index.html` loads this script at line 595, immediately before `</body>`. Everything below runs the instant it loads, with no wrapper telling it to wait for the document. That only works because, by the time the browser reaches line 595, every element already exists. Move the `<script>` tag into the `<head>` and this file would run against an empty page, find nothing, attach nothing, and report no error at all.

Line by line

Lines 1 to 6: the header, and the wrapper

```
1  /* ai.digitalise.agency · main.js
2     Scroll-driven animations: nav gloss, counter triggers, section reveals.
3     Zero external dependencies. */
4
5  (function () {
6    'use strict';
```

Listing 43: `js/main.js`, lines 1–6

Lines 1 to 3 are a comment, using the same `/* */` form as CSS. It is an accurate summary of the file, which is worth noting because a comment that describes what the code actually does is less common than it should be.

Line 5 opens a wrapper worth understanding, because it looks like noise and is not.

Jargon: Immediately invoked function expression

Writing `(function () { ... })();` defines a function and runs it once, straight away, without ever giving it a name. The point is not the running: it is the boundary. Anything declared inside is invisible outside.
Without it, the names `nav`, `animateCounter` and `observe` declared below would become global, meaning any other script on the page could see them, and any other script that happened to use the same name would collide with them. One stray global called `observe` is exactly the kind of bug that takes an afternoon to find.

Line 6 is `'use strict';`, which switches the language into a stricter mode for everything inside this wrapper.

Jargon: Strict mode

A setting that makes JavaScript refuse things it would otherwise silently allow. The most valuable one: without it, assigning to a name you never declared quietly creates a global variable, so a typo such as `navv = 3` succeeds and does nothing useful. In strict mode it throws an error and you find the typo immediately.

Lines 8 to 16: the navigation bar

```
7
8    /* -- Nav: add .scrolled glass on scroll -- */
9    const nav = document.querySelector('.nav');
10   if (nav) {
11     const onScroll = () => {
12       nav.classList.toggle('scrolled', window.scrollY > 40);
13     };
14     window.addEventListener('scroll', onScroll, { passive: true });
15     onScroll();
16   }
```

Listing 44: `js/main.js`, lines 7–16

Line 9 finds the navigation bar. `document.querySelector` takes a CSS selector, exactly the same syntax the stylesheet uses, and returns the first matching element, or the special value `null` if there is none.

Jargon: const, and why not var

`const` declares a name whose binding cannot later be pointed at something else. It is the default choice in modern JavaScript, with `let` used when a name genuinely must change. The older `var` has confusing scoping rules and appears nowhere in this file, which is a small sign of care.

Line 10 is a guard: everything only happens `if (nav)`, that is, only if something was actually found. This is the pattern that makes the whole file tolerant of a changing page. Delete the navigation bar from the HTML and this block is skipped rather than crashing.

Jargon: Truthy and falsy

JavaScript treats certain values as false when tested in a condition: `false`, `0`, an empty string, `null` and `undefined`. Everything else is treated as true. So `if (nav)` reads as "if an element was found", because `querySelector` returns `null` when it finds nothing.

Lines 11 to 13 define what to do when the page scrolls, using an arrow function.

Jargon: Arrow function

`() => { ... }` is a compact way to write a function. Here the compactness is the only benefit; the older `function () { ... }` would behave identically in this file.

Line 12 is the actual work, and it is one line doing the job of five. `classList.toggle('scrolled', condition)` adds the class when the condition is true and removes it when false. The condition is `window.scrollY > 40`, meaning the page has been scrolled more than 40 pixels from the top. The stylesheet at lines 105 to 111 defines what `scrolled` looks like, and the transition on line 103 makes the change fade rather than snap.

Line 14 registers the function to run on every scroll event. `{ passive: true }` is a performance promise to the browser: this handler will never cancel the scroll. That matters because without the promise, the browser must wait for the handler to finish before it knows whether the scroll may proceed, which can make scrolling feel sticky. It is the correct flag here and it is often forgotten.

Line 15 calls the function once immediately. This handles the case where the page is loaded already scrolled, for example when a visitor refreshes halfway down or arrives on a link ending in `#book`. Without it the bar would be transparent over content until the first scroll.

Lines 18 to 36: the counter

```
17
18    /* -- Counter animation (cubic ease-out) -- */
19    function animateCounter(el) {
20      const raw    = el.dataset.target || '0';
21      const suffix = el.dataset.suffix || '';
22      const target = parseFloat(raw.replace(/[^0-9.]/g, ''));
23      const isFloat = raw.includes('.');
24      const dec     = isFloat ? (raw.split('.')[1] || '').length : 0;
25      const dur     = 2200;
26      const start   = performance.now();
27
28      const step = (now) => {
29        const p      = Math.min((now - start) / dur, 1);
30        const eased  = 1 - Math.pow(1 - p, 3);
31        const val    = eased * target;
```

```
32        el.textContent = (isFloat ? val.toFixed(dec) : Math.round(val).toLocaleString()) +
      suffix;
33        if (p < 1) requestAnimationFrame(step);
34      };
35      requestAnimationFrame(step);
36    }
```

Listing 45: `js/main.js`, lines 17–36

This is the most substantial piece of logic in the repository, and it is worth taking slowly.

Line 20 reads the target. `el.dataset.target` is how JavaScript reads the attribute `data-target` written in the HTML: strip the `data-` prefix and what remains is a property of `dataset`. The `|| '0'` at the end supplies a default: if the attribute is missing, `dataset.target` is `undefined`, which is falsy, so the whole expression evaluates to `'0'`.

Line 21 does the same for the suffix, defaulting to an empty string.

Line 22 turns the text into a number. `raw.replace(/[^0-9.]/g, '')` deletes every character that is not a digit or a dot, and `parseFloat` then reads what is left as a number.

Jargon: Regular expression

A pattern for matching text, written between slashes. `[0-9.]` means "a digit or a dot"; putting a caret first negates it, so it means "anything that is not a digit or a dot". The `g` after the closing slash means global: replace every occurrence, not just the first.

Line 23 asks whether the original text contained a dot, which is how the code distinguishes `1.6` from `17`. Line 24 counts the digits after the dot, so that `1.6` is displayed to one decimal place throughout the animation rather than as `1.5999999`. `(raw.split('.')[1] || '')` takes everything after the dot, falling back to an empty string if there is nothing there, and `.length` counts it.

Jargon: Ternary operator

`condition ? a : b` evaluates to `a` when the condition is true and `b` otherwise. It is an `if` statement compressed into an expression, and it appears twice in this file, on lines 24 and 32.

Line 25 fixes the duration at 2200 milliseconds, 2.2 seconds. Line 26 records the start time using `performance.now()`, a clock designed for measuring durations precisely.

Lines 28 to 34 define one frame of the animation, and this is the heart of it.

`const p = Math.min((now - start) / dur, 1)` computes progress: how far through the animation we are, from 0 at the start to 1 at the end. The `Math.min(..., 1)` caps it, so that if the browser is busy and a frame arrives late, progress never exceeds 1 and the number never overshoots its target. That cap is the difference between a counter that lands on 17 and one that occasionally flashes 19 before settling.

`const eased = 1 - Math.pow(1 - p, 3)` is the easing. Linear progress feels mechanical; this curve makes the count start fast and slow dramatically as it approaches the target, which is what makes it feel like it is settling.

What the easing formula actually does

Take progress `p`, subtract it from 1, cube the result, and subtract that from 1. At `p = 0.5`, halfway through in time, the formula gives $1 - 0.5^3 = 0.875$: the counter is already 87.5% of the way to its target. The whole second half of the animation covers the last eighth of the distance. That asymmetry is the entire effect.

Line 32 renders the number. If the target had a decimal point, `toFixed(dec)` formats it to that many places. Otherwise `Math.round` makes it a whole number and `toLocaleString()` inserts

the thousands separators appropriate to the visitor's region, so a large number reads as `1,200` for a British visitor and `1.200` for a German one. The suffix is then glued on the end.

Line 33 asks for the next frame if the animation is not finished, and line 35 starts the first one.

Jargon: requestAnimationFrame

A request to run a function just before the browser's next screen repaint, about 60 times a second. Calling it repeatedly from inside itself, as here, is the standard way to run an animation: it stays in step with the display, and the browser pauses it entirely when the tab is in the background, which a timer would not.

The counter ignores the reduced-motion preference

This function animates for 2.2 seconds regardless of whether the visitor has asked their operating system to reduce motion. The stylesheet honours that preference carefully for everything it controls (lines 762 to 768), so the intent is plainly there; it simply was not carried into this file.

The fix is small and this document states it precisely so nobody has to design it:

```
const reduce = window.matchMedia('(prefers-reduced-motion: reduce)').matches;
if (reduce) {
  el.textContent = (isFloat ? target.toFixed(dec)
                            : target.toLocaleString()) + suffix;
  return;
}
```

Listing 46: A four-line fix. This code is a suggestion in this document and is NOT present in the repository.

Placed after line 26, that shows the final number immediately and skips the animation entirely, which is the correct behaviour: the information is preserved and only the movement is removed.

The metric values are hidden from anything that does not run JavaScript

The real numbers live in `data-` attributes, and the visible text is `0`. Search engines, link previews, and any visitor with JavaScript disabled therefore see a page claiming zero of everything. Writing the true value in the HTML and having the counter animate towards it from zero would remove that risk entirely at no cost to the effect.

Lines 38 to 53: the observer factory

```
37
38  /* -- Intersection observer factory -- */
39  function observe(selector, className, threshold, callback) {
40    const els = document.querySelectorAll(selector);
41    if (!els.length) return;
42
43    const io = new IntersectionObserver((entries) => {
44      entries.forEach((e) => {
45        if (!e.isIntersecting) return;
46        e.target.classList.add(className || 'visible');
47        if (callback) callback(e.target);
48        io.unobserve(e.target);
49      });
50    }, { threshold: threshold || 0.18 });
51
52    els.forEach((el) => io.observe(el));
53  }
```

Listing 47: `js/main.js`, lines 37–53

This is the best-designed thing in the repository: a small general-purpose helper that the four lines following it use to do four different jobs.

Jargon: IntersectionObserver

A browser facility that watches elements and tells you when they enter or leave the visible area. Before it existed, the only way to do this was to listen for every scroll event and measure each element's position by hand, which ran hundreds of times a second and made pages stutter. The observer does the work outside the main thread and reports only when something actually changes.

Line 40 finds every matching element. `querySelectorAll` returns a list, which may be empty, rather than `null`.

Line 41 guards against an empty list and returns early. `!els.length` reads as "if the count is zero", because `0` is falsy and `!` inverts it. This is the same defensive habit as line 10.

Lines 43 to 50 create the observer. Its callback receives a list of entries, one for each element whose visibility changed, and for each one it:

1. skips anything that is leaving rather than entering (`if (!e.isIntersecting) return`);
2. adds the class, defaulting to `visible` when the caller did not name one;
3. runs the optional extra callback, which only the counter uses;
4. stops watching that element with `io.unobserve`.

That fourth step is the one that matters most.

Why it stops watching

Every reveal in this design is meant to happen once. Without `unobserve`, an element that scrolled out of view and back would fire again, and for the metrics that would mean the numbers restarting their count every time the visitor scrolled past. Unobserving also releases the browser's bookkeeping for that element, so what starts as thirty watched elements steadily becomes zero.

Line 50 sets the threshold, defaulting to 0.18: the callback fires once 18% of the element is visible, rather than at the first pixel. Waiting for a fraction to appear is what makes the reveal feel connected to the scroll instead of triggering just off screen.

The default threshold is written in a way that cannot express zero

`threshold || 0.18` substitutes the default whenever the argument is falsy, and `0` is falsy. So a caller asking for a threshold of `0`, meaning "fire the moment a single pixel appears", would silently get `0.18` instead. The same objection applies to `className || 'visible'`, though an empty class name is not a sensible request.

No caller in this file passes `0`, so nothing is broken today. It is a latent trap of a kind worth recognising, and the modern remedy is the `??` operator, which substitutes only for `null` and `undefined` and leaves `0` alone.

Line 52 starts watching every element found.

Lines 55 to 67: the four things being watched

```
54
55    /* reveal (fade + slide) */
56    observe('.reveal', 'visible', 0.15);
57
58    /* staggered children */
59    observe('.stagger', 'visible', 0.15);
```

```
60
61    /* pipeline animation */
62    observe('.pipeline', 'visible', 0.2);
63
64    /* counters */
65    observe('.metric-num[data-target]', 'counted', 0.5, (el) => {
66      animateCounter(el);
67    });
```

Listing 48: `js/main.js`, lines 54–67

Four calls, and the whole page's reveal behaviour. The first three add the class `visible` to anything carrying `reveal`, `stagger` or `pipeline`, at slightly different thresholds. The stylesheet does everything after that.

The fourth is the different one. Its selector, `.metric-num[data-target]`, matches elements that carry the class *and* have the attribute, which is a small piece of defensiveness: a metric element without a target would be skipped rather than animated to `NaN`. Its threshold is 0.5, so the numbers only start counting when the band is half visible, and its callback runs the counter.

The class `counted` is added and styled nowhere

The fourth call passes `'counted'` rather than `'visible'`, so each metric number gains a class that no CSS rule and no other JavaScript ever reads. It is harmless, and it serves as a marker a developer could inspect, but it is worth knowing that it does nothing. Note also that the metric numbers are children of `.metrics-grid`, which is a `.stagger`, so they are already being faded in by the second call: the class here is genuinely spare.

Lines 69 to 79: smooth anchor scrolling, and the close

```
68
69    /* -- Smooth anchor scroll (book-a-call button) -- */
70    document.querySelectorAll('a[href^="#"]').forEach((a) => {
71      a.addEventListener('click', (e) => {
72        const target = document.querySelector(a.getAttribute('href'));
73        if (!target) return;
74        e.preventDefault();
75        target.scrollIntoView({ behavior: 'smooth', block: 'start' });
76      });
77    });
78
79  })();
```

Listing 49: `js/main.js`, lines 68–79

Line 70 finds every link whose destination starts with a hash. `a[href^="#"]` is an attribute selector, and `^=` means "starts with". On this page that matches three links: the navigation's "Book a call", the hero's "Book a call", and the hero's "See the work".

Line 72 looks up the destination by using the link's own `href` as a selector, which works because `#book` is valid CSS meaning "the element with id book".

Line 73 returns early if the destination does not exist, leaving the browser to handle the click normally. Line 74 then cancels the browser's default jump, but only after the destination has been confirmed, which is the correct order: cancel first and a broken link would do nothing at all instead of at least trying.

Line 75 performs the scroll. `block: 'start'` aligns the destination with the top of the window.

This whole block is redundant, and it carries a latent crash

`css/site.css` line 28 already sets `scroll-behavior: smooth` on the document, which makes every anchor link glide without any JavaScript at all. These eleven lines reproduce that.

They also introduce a failure the CSS version does not have. A link written as `href="#"`, which is a common way to write a placeholder link, would reach line 72 as `document.querySelector('#')`. That is not a valid selector, so the browser throws a `SyntaxError`, and because the error happens inside a click handler it would break that click and leave an error in the console.
No such link exists in the page today, so this is latent rather than live. The honest summary is that the safest change to this file would be to delete lines 69 to 77 entirely: it would remove a redundancy and a hazard in one edit, and nothing visible would change.

Line 79 closes and immediately runs the wrapper opened on line 5.

What it connects to

Named by: `new-design-reference/index.html`, line 595, and nothing else.

Names: no file at all. It imports nothing and fetches nothing.

Its entire interface with the rest of the project is seven strings, and they are worth listing because they are the contract nothing checks:

String	Used at	Where it must also exist
`.nav`	line 9	`index.html` line 23, `site.css` line 95
`scrolled`	line 12	`site.css` line 105
`.reveal`	line 56	`index.html` (five places), `site.css` line 78
`.stagger`	line 59	`index.html` (four places), `site.css` line 86
`.pipeline`	line 62	`index.html` line 223, `site.css` line 381
`visible`	lines 56–62	`site.css` lines 83, 92, 400
`.metric-num`	line 65	`index.html` (four places), `site.css` line 343

Table 4: Every point of contact between this file and the other two. Break any one of these strings in one place and the page keeps working, minus one behaviour, with no error reported anywhere.

How it breaks

- **Load it in the `<head>` instead of at the end of the body** and all three behaviours vanish silently, since every lookup runs immediately and finds nothing.
- **Rename a class in the HTML** and the corresponding behaviour stops, again silently, because of the early-return guards.
- **Delete this file** and the page is static, with four zeros in the metrics band. That last part is the only real damage.
- **Add a link with `href="#"`** and clicking it throws a `SyntaxError` from line 72.
- **A visitor who has asked for reduced motion** still gets the counting animation, because this file never checks.
- **An ancient browser without `IntersectionObserver`** would throw on line 43 and lose all four reveal behaviours, which would leave every `.reveal` element permanently invisible were it not for the CSS reduced-motion block. Every browser in current use supports it, so this is a historical note, not a live concern.

Honest findings for this file

The counter ignores `prefers-reduced-motion`; the real metric values are invisible without JavaScript; the smooth-scroll block duplicates a CSS feature and carries a latent `SyntaxError`; the `counted` class is inert; and the `||` defaults on line 47 and 50 cannot express a falsy argument. Against

those: the file is genuinely well written, with consistent guards, a sensible shared helper, correct use of `unobserve` and `passive`, and no dependencies at all.

7.4 `new-design-reference/netlify.toml`

TOML, 11 lines. Instructions for a hosting service called Netlify: how to publish the folder, and four security headers to send with every response.

Where this came from: new-design-reference/netlify.toml

Hand-written, added in the first commit `0d77d51` and never edited. The structure is the standard shape from Netlify's own documented configuration format, which is what any developer writing one of these would produce; that is convention rather than copying, and no specific source is claimed. **Confidence: certain for hand-written, no provenance evidence found for any particular template.**

Why it exists

Today, for no reason at all: nothing uses it.

The honest reading of the history is that the site was first expected to be hosted on Netlify, which is a common choice for a static page like this, and the plan changed three commits later. Commit `2c66a7a` on 2026-07-28 is titled in part "switch deploy to VPS rsync", which is the moment Netlify stopped being the destination. This file was not removed at that point and has sat unused since.

Jargon: TOML

A file format for configuration, designed to be obvious to read. A name in square brackets starts a section; `name = "value"` sets a setting. Double square brackets, as on line 5, mean "another entry in a list of these", so a file can carry several header rules.

Line by line

```
1  [build]
2    publish = "."
3    command = ""
4
5  [[headers]]
6    for = "/*"
7    [headers.values]
8      X-Frame-Options = "DENY"
9      X-Content-Type-Options = "nosniff"
10     Referrer-Policy = "strict-origin-when-cross-origin"
11     Permissions-Policy = "camera=(), microphone=(), geolocation=()"
```

Listing 50: `netlify.toml`, lines 1–11, the complete file

Lines 1 to 3 describe how to build the site. `publish = "."` means publish this directory itself, and `command = ""` means there is no build step to run first. Those two lines are the whole of the "no build step" decision, stated to a hosting provider.

Line 5 opens a headers rule and line 6 applies it to `/*`, meaning every path on the site.

Jargon: HTTP header

An extra line of information a web server sends alongside a file, which the visitor never sees. Some describe the file; the four below are instructions to the browser about what it is allowed to do with the page.

Line 8, `X-Frame-Options = "DENY"`, forbids any other site from embedding this page inside a frame. That defends against clickjacking, where an attacker loads your real page invisibly over their own and tricks a visitor into clicking your buttons while believing they are clicking something else.

Line 9, `X-Content-Type-Options = "nosniff"`, tells the browser to believe the declared type of every file rather than guessing from its contents. Guessing is a historical browser behaviour that can be manipulated into treating an uploaded image as a script.

Line 10, `Referrer-Policy = "strict-origin-when-cross-origin"`, limits what is disclosed when a visitor follows a link away from the page. Within this site the full address is passed; to another site only the domain is sent, and to an insecure site nothing at all. This is the modern default and a sensible privacy choice.

Line 11, `Permissions-Policy`, switches off three device capabilities for this page entirely: camera, microphone and location. The empty parentheses mean "nobody, not even this page". Since the page is static text and images, it needs none of them, and turning them off means a future compromise cannot quietly request them.

These four headers are good practice and they are not in effect

Every one of these is a sensible, defensive choice, and the file they live in is read by a service this project does not use. The page is served by Caddy on the agency VPS, which never sees this file.
Moving these four lines into the Caddy configuration on the server would take a few minutes and would make them real. As things stand, the repository looks like it has thought about security headers and the live site has none of them.

What it connects to

Nothing in this repository refers to it. Netlify appears nowhere else in the tree and no workflow reads it. The deploy workflow does not even exclude it, so if that workflow were ever restored, this file would be copied to the VPS where Caddy would ignore it.

How it breaks

It cannot break, because nothing reads it. The realistic failure is the opposite: a future maintainer reads this file, concludes the site sends these headers, and builds on an assumption that is false.

An unused configuration file is worse than no configuration file

This is a small file and a real trap. It states four security guarantees that the live site does not provide, and there is nothing in the repository to indicate it is inert. Either delete it, or move its contents to the Caddy configuration and delete it. Leaving it is the only option with no upside.

7.5 `new-design-reference/assets/ASSETS.md`

Markdown, 33 lines. A checklist of the twelve files somebody would need to produce before this design could be considered finished, and three notes about placeholder content to replace.

Where this came from: new-design-reference/assets/ASSETS.md

Hand-written, added in the first commit `0d77d51` alongside the design it describes, and never edited since. Written by the same person in the same sitting as the HTML, which is evident from the fact that it references the exact class names (`pt1` through `pt6`, `pa1`, `pa2`) used in that file. **Confidence: high, from the commit history and the internal references.**

Why it exists

It is a handover note. The design was built with placeholder visuals, and this file records precisely what would replace each one and what HTML change each replacement needs. As an artefact it is genuinely good practice: it converts "this is not finished" from a feeling into a list.

Jargon: Markdown

A way of writing formatted text in a plain file, using punctuation instead of menus: `#` starts a heading, backticks mark code, and pipe characters draw a table. It is designed to be readable as plain text and is the standard format for notes kept alongside code.

Line by line

```
1  # Assets checklist: ai.digitalise.agency
2
3  Drop real assets here, then update the HTML as noted.
4
5  ## Required before launch
6
7  | File | Size | Where used | HTML change needed |
8  |---|---|---|---|
9  | `videos/hero-1.mp4` | under 8MB | Hero background tiles | Swap .tile-blue .tile-thumb div
     for `<video autoplay muted loop playsinline>` |
10 | `logos/client-1.svg` | any | Proof section | Replace `pa1`/`pa2` avatar initials with `<img
     >` |
11 | `logos/client-2.svg` | any | Proof section | Same |
12 | `logos/client-3.svg` | any | Proof section | Same |
13 | `thumbs/port-1.jpg` | 800x450px | Portfolio card 1 | Add `style="background-image:url('/
     assets/thumbs/port-1.jpg')"` to `.pt1` |
14 | `thumbs/port-2.jpg` | 800x450px | Portfolio card 2 | Same pattern for `.pt2` through `.pt6`
     |
15 | `thumbs/port-3.jpg` | 800x450px | Portfolio card 3 | |
16 | `thumbs/port-4.jpg` | 800x450px | Portfolio card 4 | |
17 | `thumbs/port-5.jpg` | 800x450px | Portfolio card 5 | |
18 | `thumbs/port-6.jpg` | 800x450px | Portfolio card 6 | |
19 | `favicon.svg` | 32x32 | Browser tab | Add `<link rel="icon" href="/assets/favicon.svg">` to
     `<head>` |
20 | `og-image.png` | 1200x630px | Social share preview | Update `og:image` meta tag |
```

Listing 51: `ASSETS.md`, lines 1–21

Line 1 is the title, and line 3 the instruction: put the real files in this folder, then change the HTML as the table says.

Lines 7 and 8 open a table, with line 8 being the separator row that Markdown requires between a table's headings and its body.

The twelve rows are the substance. Reading them tells you exactly what the design is pretending to have:

- **One video**, under 8 megabytes, to play inside the hero tiles instead of the static icons. The suggested tag carries `autoplay muted loop playsinline`, and the `muted` is not optional: browsers refuse to autoplay video with sound, so an unmuted autoplaying video simply does not start. `playsinline` stops iPhones taking the video full screen.
- **Three client logos**, to replace the coloured circles with initials in the proof section. Note the row says `pa1`/`pa2` and there are three logos and three avatars (`pa1` to `pa3`), so the instruction is one short.
- **Six portfolio thumbnails** at 800 by 450, which is exactly the 16 by 9 shape the stylesheet reserves with `aspect-ratio` on line 473.

- **A favicon**, with the exact tag needed to install it.
- **A social preview image** at 1200 by 630, the standard Open Graph size, to fix the missing `og:image` noted in section 7.1.

None of the twelve exist

The `assets/` folder contains exactly one file: this checklist. Every asset it calls "required before launch" is absent, which means the design was never finished to the standard its own author set for it. The portfolio thumbnails are still gradients, the client logos are still lettered circles, and the page has neither a favicon nor a social preview image.

```
21
22 ## Numbers to update (index.html metrics band)
23
24 Search for `data-target` in `index.html` and replace all four values with real numbers
       verified from Gantry / n8n data.
25
26 ## Booking link
27
28 Search for `YOUR_CAL_LINK` in `index.html` and replace with the actual Cal.com or Calendly URL.
29 For a full inline calendar embed, replace the `<a>` tag with the Cal.com embed snippet from
       cal.com/docs/embeds.
30
31 ## Client proof section
32
33 Three `<!-- UPDATE -->` comments in the proof section mark where real client names and
       companies go.
```

Listing 52: `ASSETS.md`, lines 21–33

Line 24 is the most interesting line in the file, because of one phrase: the four metric numbers are to be replaced with "real numbers verified from Gantry / n8n data". Gantry is another project in this workspace, the agency's content publishing system, and n8n is the automation tool behind several of these projects. So the author's intention was that the numbers on this marketing page would be traceable to the agency's own operational systems rather than estimated. That is a higher standard than most marketing pages hold themselves to, and it makes the invented hero tile numbers in `index.html` more surprising rather than less.

Line 28 tells the reader to search for `YOUR_CAL_LINK`.

Two of this file's three instructions are stale

Searching `index.html` for `YOUR_CAL_LINK` finds nothing. The literal placeholder was replaced on 2026-07-28 by commit `4dffc12`, which set the link to `cal.com/davonex/30min`. So somebody following this instruction searches, finds nothing, and has no way to tell whether the job is done or the instruction is wrong. The answer is that the job is half done: the placeholder text is gone and the link still points at another project's calendar.

The metrics instruction on line 24 is stale in the same way: commit `2c66a7a` claims to have replaced all placeholder content with real agency data, and the four values do look like considered numbers rather than round placeholders. Yet the `UPDATE` comment sits in `index.html` at line 187 and this instruction sits here, both saying the work is outstanding.

Only the third instruction, on line 33 about the client proof section, is verifiably still true: the avatar initial on line 430 of `index.html` is a leftover.

What it connects to

It describes `new-design-reference/index.html` and `new-design-reference/css/site.css` by name and by class, and it is the only file in the repository that documents the project's intent. Nothing reads it: it is for people.

The deploy workflow explicitly excludes it from publication (line 82 of the workflow), which is the right instinct, and that exclusion no longer matches its path. See section 7.6.

How it breaks

As a document it cannot break. As a source of truth it already has: two of its three instructions describe a state of the code that no longer exists, which is the characteristic failure of documentation kept beside code and not updated with it.

7.6 `.github/workflows-disabled/deploy.yml.disabled`

YAML, 103 lines. The automation that used to publish this site: it watched for pushes, checked for a marker in the commit message, copied the files to the agency's server over an encrypted connection, and then checked that the site still answered. It is currently switched off.

Where this came from: .github/workflows-disabled/deploy.yml.disabled

Hand-written, added in commit `d49e034` on 2026-07-28 at the path `.github/workflows/deploy.yml`, rewritten in `2c66a7a` the same day when the deploy target changed from Netlify to the VPS, fixed three times on 2026-07-29, and finally renamed to its present path in `da508a7`.
Its shape is shared with the sibling project `salmanadnan-com`, whose workflow was written in the same week and carries the same retry loops and the same `[deploy]` marker check. Neither file says which came first, and this document does not guess. The `[deploy]` convention itself is workspace-wide and predates both. **Confidence: certain for hand-written and for the rename history, which is recorded in git; the relationship to the sibling repository is stated as a resemblance and not as a lineage.**

Why it exists

To remove a manual step. Without it, publishing means somebody connecting to the server and copying files by hand, which is slow and easy to get wrong. With it, publishing is a consequence of writing a commit message in a particular way.

Jargon: YAML

A file format for configuration, where structure is expressed by indentation rather than by brackets. Two spaces of indent mean "this belongs to the line above". It is unforgiving about indentation, which is its main drawback: a single misplaced space changes the meaning.

Jargon: GitHub Actions, and a workflow

A service that runs jobs on GitHub's computers when something happens in a repository. The jobs are described in a workflow file in `.github/workflows/`. Each job runs on a fresh, empty machine, which is why the file below has to install its own SSH key every time.

Line by line

Lines 1 to 19: the header comment and the trigger

```
1  # ai.digitalise.agency: deploy to VPS when commit subject starts with [deploy].
2  #
3  # A push to main always checks the marker. It DEPLOYS only when the commit
4  # subject line STARTS with [deploy], matching every other agency repository.
5  # Mentioning the marker anywhere else in the message does nothing.
6  #
7  # The site is served by Caddy from /root/ai.digitalise.agency on the VPS.
8  # Caddy picks up the new files immediately, no restart needed.
9  #
10 # Required repository secrets:
11 #   VPS_HOST     46.202.194.85
12 #   VPS_USER     deploy
13 #   VPS_SSH_KEY  private key for that user
14 name: Deploy ai.digitalise.agency
15
16 on:
17   push:
18     branches: [main]
```

Listing 53: `deploy.yml.disabled`, lines 1–19

Lines 1 to 13 are a comment block, and it is the best documentation in the repository. In YAML a `#` starts a comment. It states the marker rule twice and in capitals, which tells you it had caught somebody out; it names the serving path and the fact that Caddy needs no restart; and it lists the three secrets the workflow needs.

The server's address is written in the comment that explains it is a secret

Line 11 records `VPS_HOST 46.202.194.85` in plain text, immediately under the heading "Required repository secrets". Whatever the value of keeping the address in GitHub's secret store, writing it here defeats it.
Proportion matters: an IP address is not a credential, this repository is private, and the server is reachable on the public internet anyway. The genuine cost is that it is now in the git history permanently, so if the server is ever replaced this line becomes stale and misleading rather than merely redundant. The private key, which is the thing that actually matters, is correctly absent.

Line 14 names the workflow, which is the label shown in GitHub's interface.

Lines 16 to 18 set the trigger: run on every push to the branch `main`, and nothing else. Not on pull requests, not on a schedule, not manually. That last absence is worth noting: there is no `workflow_dispatch`, so there is no way to publish without making a commit.

Lines 20 to 38: the marker check

```
19
20 jobs:
21   check-deploy:
22     name: Check Deploy Marker
23     runs-on: ubuntu-latest
24     outputs:
25       should_deploy: ${{ steps.check.outputs.should_deploy }}
26     steps:
27       - id: check
28         env:
29           COMMIT_MSG: ${{ github.event.head_commit.message }}
30         run: |
31           SUBJECT="${COMMIT_MSG%%$'\n'*}"
32           if [[ "$SUBJECT" == "[deploy]"* ]]; then
33             echo "should_deploy=true" >> "$GITHUB_OUTPUT"
34             echo "Deploy marker found on the subject line."
35           else
```

```
36              echo "should_deploy=false" >> "$GITHUB_OUTPUT"
37              echo "No deploy marker: skipping deploy."
38            fi
```

Listing 54: `deploy.yml.disabled`, lines 19–38

The first job exists only to answer a yes-or-no question, and it runs on a fresh Ubuntu machine to do it. Lines 24 and 25 declare that this job produces an output called `should_deploy`, which the second job will read.

Line 28 passes the commit message into the script as an environment variable rather than pasting it directly into the script text. That distinction is a genuine security measure.

Why the commit message goes through an environment variable

If line 31 had been written as `SUBJECT="${{ github.event.head_commit.message }}"`, GitHub would paste the commit message directly into the script before the shell ever sees it. Anyone who could write a commit message could then write shell commands into it and have them run on the machine holding the deploy key.
Passing it as an environment variable and reading it as `$COMMIT_MSG` means the message is data, never code. This is a well-known class of vulnerability in GitHub Actions, and this file gets it right.

Line 31 extracts the subject line: `${COMMIT_MSG%%$'\n'*}` is shell syntax meaning "delete the longest match of a newline followed by anything, from the end", which leaves everything before the first newline.

Line 32 is the test itself. `"$SUBJECT" == "[deploy]"*` compares with a trailing wildcard, which is why the marker must *begin* the subject.

Lines 33 and 36 write the answer to `$GITHUB_OUTPUT`, a file GitHub reads to collect a step's outputs. Lines 34 and 37 print a human-readable explanation into the log, which is a small kindness to whoever is wondering why nothing deployed.

Lines 40 to 63: starting the deploy, and the SSH setup

```
39
40    deploy:
41      name: Deploy to VPS
42      runs-on: ubuntu-latest
43      needs: check-deploy
44      if: needs.check-deploy.outputs.should_deploy == 'true'
45      steps:
46        - uses: actions/checkout@v4
47
48        - name: Authorize SSH
49          env:
50            KEY: ${{ secrets.VPS_SSH_KEY }}
51            HOST: ${{ secrets.VPS_HOST }}
52          run: |
53            mkdir -p ~/.ssh && chmod 700 ~/.ssh
54            printf '%s\n' "$KEY" > ~/.ssh/deploy_key
55            chmod 600 ~/.ssh/deploy_key
56            for i in 1 2 3 4 5; do
57              if ssh-keyscan -T 10 -H "$HOST" >> ~/.ssh/known_hosts 2>/tmp/keyscan.err; then
58                [ -s ~/.ssh/known_hosts ] && break
59              fi
60              echo "ssh-keyscan attempt $i failed: $(cat /tmp/keyscan.err 2>/dev/null)"
61              sleep 5
62            done
63            [ -s ~/.ssh/known_hosts ] || { echo "::error::could not fetch the host key"; exit 1;
        }
```

Listing 55: `deploy.yml.disabled`, lines 39–63

Line 43 makes this job wait for the first, and line 44 runs it only if the answer was yes. Everything below is therefore skipped entirely on an ordinary commit.

Line 46 checks out the repository onto the machine, using GitHub's own action. Without this the machine would be empty.

Lines 53 to 55 install the private key: create the `.ssh` folder, write the key into it, and set permissions so only the owner can read it. `chmod 600` is not optional; SSH refuses to use a key file that other users can read, and its error message when it does is famously unhelpful.

Jargon: SSH key pair, and the host key

Connecting securely involves two separate keys and confusing them is common. The *deploy key* here is the private half of a pair proving *we* are allowed in; its public half sits on the server. The *host key* is the server's own identity, which the client checks to be sure it is talking to the right machine and not an impostor. Lines 53 to 55 install the first. Lines 56 to 63 fetch the second.

Lines 56 to 62 are the retry loop, and they exist because this step failed repeatedly. `ssh-keyscan` asks the server for its host key, with a 10-second timeout, appending the result to `known_hosts` and any complaint to a temporary file. `[ -s /.ssh/known_hosts ]` then tests that the file is non-empty, because `ssh-keyscan` can succeed as a command while producing nothing at all. That distinction is the subtle part: checking the exit code alone would not have been enough.

Line 63 fails the whole job with a message if five attempts produced nothing. `::error::` is GitHub's syntax for marking a line in the log as an error so it surfaces in the interface.

Why fetching the host key each time is a real trade-off

Fetching a server's identity at the moment you connect, and trusting whatever comes back, means you are not actually verifying anything: an attacker positioned between the runner and the server could supply their own. The secure alternative is to store the expected host key as a secret and compare.

That is a real weakness and it is a widespread convention, and the sibling project `passbolt-digitalise` in this workspace does it the stricter way, keeping a `.github/known_hosts` file in the repository. Two projects in one workspace solving the same problem two ways is worth knowing about.

Lines 65 to 83: copying the files

```
64
65      - name: Sync files
66        env:
67          HOST: ${{ secrets.VPS_HOST }}
68          USER: ${{ secrets.VPS_USER }}
69        run: |
70          retry() {
71            for attempt in 1 2 3; do
72              "$@" && return 0
73              echo "attempt $attempt failed: $*"
74              [ "$attempt" -lt 3 ] && sleep $(( attempt * 15 ))
75            done
76            return 1
77          }
78          retry rsync -az --no-owner --no-group --omit-dir-times --delete \
79            -e "ssh -i ~/.ssh/deploy_key -o BatchMode=yes -o ConnectTimeout=25" \
80            --exclude '.git' \
81            --exclude '.github' \
82            --exclude 'assets/ASSETS.md' \
83            ./ "$USER@$HOST:/root/ai.digitalise.agency/"
```

Listing 56: `deploy.yml.disabled`, lines 64–83

Lines 70 to 77 define a small shell function that runs whatever it is given, up to three times, waiting longer between each attempt: 15 seconds, then 30. `"$@"` means "all the arguments, run as a command". This is the second retry loop in the file, and like the first it is evidence of a step that failed in practice.

Line 78 is the actual copy. Each flag earns its place:

Flag	What it does
`-a`	archive mode: recurse, and preserve timestamps, permissions and links
`-z`	compress during transfer
`-no-owner`	do not try to set file ownership on the far end
`-no-group`	do not try to set group ownership either
`-omit-dir-times`	do not try to set directory timestamps
`-delete`	remove files on the server that no longer exist here

Table 5: The `rsync` flags, and why each is present.

The three `-no`/`-omit` flags are all working around the same thing: the `deploy` user is not the owner of the target directory and lacks the privilege to change ownership or directory timestamps. Without them `rsync` reports errors and fails. These flags are the surviving trace of the problem that commit `f698421` was fixing.

`-delete` deserves respect: it makes the server an exact mirror, so removing a file here removes it there. That is usually what you want and it is also the flag that turns a mistaken source path into a wiped site.

Line 79 tells `rsync` how to connect: use this key, never prompt for a password (`BatchMode=yes`, essential because there is no human to answer), and give up after 25 seconds.

Lines 80 to 82 exclude three things from the copy: the git history, the workflow folder, and the internal checklist.

Line 83 is the source and destination. `./` means the current directory's *contents*, and the trailing slash is significant: without it, `rsync` would copy the directory itself into the target, producing a nested folder.

This line would now break the live site

`./` was the right source when `index.html` sat at the repository root. Since commit `da508a7` moved everything down one level, `./` now contains a single folder called `new-design-reference`. Restoring this workflow unchanged would therefore copy that folder to the server and, because `-delete` is set, remove the real site's files in the same operation.
The correct source after the move would be `new-design-reference/`. The same commit that moved the files disabled this workflow, so the two are consistent; the trap only exists for somebody who re-enables one without reading the other.

The exclusion for the checklist no longer matches

`-exclude 'assets/ASSETS.md'` was written when that path was correct. The file now lives at `new-design-reference/assets/ASSETS.md`, which this pattern does not match, so a restored workflow would publish the internal checklist, including its line about replacing placeholder numbers, to the public internet.

Lines 85 to 103: verifying, and cleaning up

```
84
85        - name: Verify
86          env:
87            HOST: ${{ secrets.VPS_HOST }}
88            USER: ${{ secrets.VPS_USER }}
```

```
89          run: |
90            for i in $(seq 1 12); do
91              code=$(ssh -i ~/.ssh/deploy_key -o BatchMode=yes "$USER@$HOST" \
92                'curl -sf -o /dev/null -w "%{http_code}" --max-time 10 https://ai.digitalise.
        agency/' || echo 000)
93              case "$code" in
94                2*|3*) echo "Site answering with $code after ${i} check(s)."; exit 0 ;;
95              esac
96              echo "waiting, got $code ($i/12)"
97              sleep 5
98            done
99            echo "::warning::site did not respond 2xx/3xx; check Caddy on the VPS"
100
101       - name: Clean up key
102         if: always()
103         run: rm -f ~/.ssh/deploy_key
```

Listing 57: `deploy.yml.disabled`, lines 84–103

The verification step asks, up to twelve times over about a minute, whether the site is answering. It does this by connecting to the server and running `curl` *there* rather than from the runner, which tests the server's own view of itself.

The `curl` flags: `-s` quiet, `-f` treat an error status as a failure, `-o /dev/null` throw the page away, `-w "%{http_code}"` print just the status number, and a 10-second limit. The `|| echo 000` supplies a fallback so that a failed connection yields the harmless string `000` instead of an empty one.

Line 93 accepts any status beginning with 2 or 3, meaning success or redirect, and exits successfully.

A failed verification is only a warning

Line 99 uses `::warning::` rather than `::error::`, and the step does not exit with a failure code. So if the site never came back, the workflow would still finish green with a warning buried in the log. Given that the preceding step ran `rsync -delete` against the live site, a failure here is the one moment you would want a loud red failure and an email. This is the clearest correctness weakness in the workflow, and it is one word to fix.

Lines 101 to 103 delete the private key from the runner. `if: always()` makes this run even when an earlier step failed, which is the point: a key must not be left behind on a machine because something else went wrong. In practice GitHub destroys the machine anyway, so this is belt and braces, and it is the right instinct.

What it connects to

Reads: three GitHub secrets (`VPS_SSH_KEY`, `VPS_HOST`, `VPS_USER`) and the commit message of the push that triggered it.

Writes: the contents of `/root/ai.digitalise.agency/` on the VPS at `46.202.194.85`, which Caddy serves at `https://ai.digitalise.agency/`.

Excludes: `.git`, `.github` and `assets/ASSETS.md`.

Runs when: never, currently. It sits in `.github/workflows-disabled/` with the extension `.disabled`, and GitHub only reads `.github/workflows/*.yml`.

How it breaks

- **Restored unchanged**, it would copy the wrong directory and, with `-delete`, take down whatever is currently served.

- **A missing or wrong** `VPS_SSH_KEY` fails at the `rsync` step after three attempts. This is what commit `ed5c154` fixed.
- **A** `deploy` **user without permission to traverse into** `/root` fails the same way. This is what `f698421` fixed, and it is the least obvious of the three: the user can hold the key, connect successfully, and still be unable to reach the target directory.
- **GitHub's runner being refused by the server** fails the host-key fetch, which is what both retry loops exist for.
- **A commit whose subject does not start with** `[deploy]` does nothing at all, by design, and the log says so.
- **The site failing to come back** produces a warning and a green tick, which is the failure mode worth fixing first.

Honest findings for this file

The server address is recorded in a comment describing it as a secret; the `rsync` source path and the `ASSETS.md` exclusion are both stale after the move; the host key is trusted on first sight rather than pinned, unlike the sibling `passbolt-digitalise` repository; and a failed post-deploy check produces a warning rather than a failure. Against those: the commit message is handled safely as data rather than as code, both retry loops are well built, the key is cleaned up unconditionally, and the comment block at the top is the most useful documentation in the repository.

8 Generated and third-party files

There are none. Every one of the six tracked files was written by hand by a person. There is no lock file, no minified bundle, no vendored dependency tree, no binary asset, and no file carrying a generator's banner. This was checked mechanically rather than assumed:

```
$ git grep -lIE '(DO NOT EDIT|auto-?generated|@generated|Generated by)'
(no output)
```

Listing 58: The check that was run, and its result

So the tier 2 count in Appendix A is zero, and every file is walked line by line.

9 Configuration and secrets

Nothing in this repository is configurable at run time. There is no settings file, no environment file, and no `.env.example`. The page is entirely fixed: to change what it says, you edit the HTML.

Three values are effectively configuration that has been written directly into the files, which is worth knowing because it is where you would look to change them:

Value	Where it is	What it controls
`https://cal.com/davonex/30min`	`index.html`, line 565	the booking calendar that opens
`hello@digitalise.agency`	`index.html`, line 590	the contact address in the footer
The colour tokens	`css/site.css`, lines 5–25	the whole colour scheme

Table 6: Values that behave like settings but are written into the source.

9.1 Secrets

The repository contains **no passwords, keys or tokens**, and it should not: a page that runs entirely in a visitor's browser can keep no secret, because the visitor can read every byte of it.

Three secrets are *referred to* by the deploy workflow, and stored in GitHub rather than here: `VPS_HOST`, `VPS_USER` and `VPS_SSH_KEY`. Section 12 covers how they are used.

The server's IP address is written in a comment

Line 11 of the deploy workflow records `VPS_HOST 46.202.194.85` in a comment, next to the note that it is supposed to be a repository secret. Storing the address in a comment defeats the point of storing the value in a secret. This is a small exposure rather than a large one, because an IP address is not a credential and this repository is private, but it is inconsistent with its own stated design, and the same address is reachable from anywhere on the internet.

10 Dependencies

The project has **no build-time dependencies at all**: no `package.json`, no `requirements.txt`, nothing to install. That is unusual enough to be worth stating plainly rather than leaving as an absence.

It has exactly two run-time dependencies, both of them services on other people's computers, both reached by the visitor's browser rather than by the project:

Dependency	Reached from	What it is for
Google Fonts	`index.html`, lines 13–15	downloads the Space Grotesk and Inter typefaces
cal.com	`index.html`, line 565	shows the calendar when "Book a call" is pressed

Table 7: The two external services the page depends on.

Fonts are loaded from Google, which the agency's main site deliberately does not do

The agency's main website (the `digitalise-website` project in this same workspace) self-hosts its fonts, meaning it serves the font files from its own server. This page instead fetches them from `fonts.googleapis.com` on every visit. That has two consequences: every visitor's browser makes a request to Google, which is a privacy and consent question in some jurisdictions, and the page's appearance depends on a third party staying up.

The page does degrade gracefully if Google is unreachable, because both font definitions list fallbacks (`ui-sans-serif`, `system-ui`, `sans-serif`), so text still appears in whatever the visitor's device provides. The layout would shift, but nothing would disappear.

11 How to run it

There is nothing to install and nothing to start. Two ways work:

```
# From the repository root
xdg-open new-design-reference/index.html      # Linux
open      new-design-reference/index.html     # macOS
```

Listing 59: The simplest way: open the file directly

This works, with one caveat that will confuse a beginner. The page asks for its stylesheet at `/css/site.css`, with a leading slash. A leading slash means "from the very top of this site". When the file is opened directly off a disk, the very top is the root of your hard drive, so the browser looks for `/css/site.css` there, does not find it, and renders the page with no styling at all: black text on white, everything stacked in a column.

To see it as intended, serve the folder over HTTP instead:

```
cd new-design-reference
```

```
python3 -m http.server 8000
# then open http://localhost:8000/ in a browser
```

Listing 60: The way that actually looks right

Why the leading slash matters so much

`/css/site.css` means "start at the top of the site". `css/site.css`, without the slash, would mean "start from wherever the current page is". The second form would have worked both on a server and off a disk. The first form was chosen, which is correct for the live site and inconvenient for local inspection. This one character is the single most likely thing to make a newcomer think the project is broken when it is not.

11.1 What you should see

A full-screen dark navy hero with the headline "AI systems built to grow while you sleep", eight small cards drifting in and out behind it, and below that, in order: four numbers that count up as they scroll into view, a five-step horizontal pipeline, six project cards, three client cards, six service tiles, a booking panel, and a one-line footer.

12 The agent-facing surface

Several projects in this workspace are operated through Claude Code and ship their own commands, skills and hooks. **This project ships none.** That was checked rather than assumed:

```
$ find . -maxdepth 2 \( -name 'CLAUDE.md' -o -name 'skills.md' \)
(no output)
$ find .claude -type f 2>/dev/null
(no output)
```

Listing 61: The check, and its result

There is no `.claude/` folder, no `CLAUDE.md`, no skill and no hook.

No CLAUDE.md and no README means the repository explains nothing about itself

There is no README either. A person cloning this repository is given six files and no statement of what it is, whether it is live, or why everything sits inside a folder called `new-design-reference`. The only explanation anywhere is the message on commit `da508a7`, which is invisible unless you know to run `git log`. For a repository whose central fact is "this is parked, the live site is elsewhere", that fact belongs in a README where it cannot be missed. This is the most consequential gap in the project and it would take five minutes to close.

The document you are reading, and its companion Brief, are produced by a workspace-level command, `/explain-projects`, which lives outside this repository and writes into `docs/explainer/`. That folder is the only part of this repository that command owns.

13 Deployment

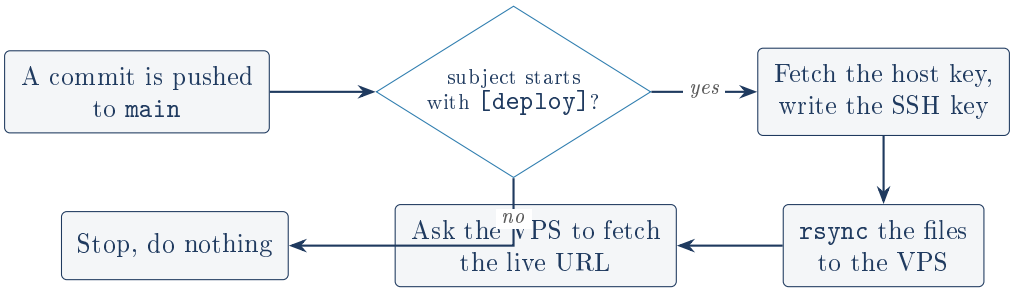


Figure 7: The publishing pipeline as written in the workflow file. It is drawn in the past tense on purpose: this pipeline is switched off and has been since 2026-07-29.

The mechanism is explained line by line in section 7.6. The short version: a push to the `main` branch checks whether the commit's first line begins with the literal text `[deploy]`. If it does, the files are copied to the VPS with `rsync` and the workflow then asks the VPS to fetch the live address to confirm the site still answers. If it does not, nothing happens.

The `[deploy]` marker must start the subject line

Every repository in this workspace shares this convention, and it catches people out. Writing "fix the header and `[deploy]`" does nothing at all: the check is `"$SUBJECT" == "[deploy]"*`, which tests the *beginning* of the string. The marker must be the first thing in the commit message.

13.1 Why it is disabled, and what would happen if it were not

The workflow is disabled twice over, as section 4 explained. If somebody restored it by moving the file back and renaming it, it would not work, and it is worth knowing why before anybody tries.

The `rsync` command copies `./`, the repository root, to `/root/ai.digitalise.agency/` on the server. When the workflow was written, `index.html` was at that root. Since commit `da508a7` it is one level down. So a restored workflow would copy a folder called `new-design-reference` to the server, the server would look for a home page at the top and find none, and the live site would break. The same commit that moved the files is the one that disabled the workflow, so the two changes are consistent with each other; the trap is only there for somebody who re-enables one without the other.

One exclusion is now stale as well

The `rsync` command excludes `assets/ASSETS.md` so the internal checklist is not published. After the move, that file is at `new-design-reference/assets/ASSETS.md`, which the exclusion pattern no longer matches. Re-enable the workflow unchanged and the checklist, including the line about replacing placeholder numbers, would be published to the public internet.

14 Honest findings

Everything here is also stated at the point in the walkthrough where it arises. It is collected once so it can be read as a list.

14.1 Findings about correctness

1. **The live site's canonical link points at a different domain.** The page served at ai.digitalise.agency declares salmanadnan.com as its canonical URL, which asks search engines to index the other site instead. Verified live on 2026-08-11. Not fixable from this repository, but whoever fixes it should know.
2. **A restored deploy workflow would break the live site**, because the files moved and the `rsync` source path did not. Section 13.1.
3. **The `rsync` exclusion for `ASSETS.md` no longer matches** its path, so re-enabling the workflow would publish the internal checklist.

14.2 Findings about dead or contradictory code

4. **Two colour tokens are defined and never used.** `-accent2` (an orange) and `-shadow` are declared in `site.css` and read zero times. Counted mechanically: `grep -c 'var(-accent2)'` returns 0.
5. **The rule `.metric-num .accent` styles nothing.** No element in `index.html` carries the class `accent`.
6. **One responsive rule contradicts itself.** In the 900-pixel breakpoint, tile 3 is shown by one rule (line 748) and hidden by another five lines later (line 751). The hide wins, so the show is dead.
7. **`.tile-lines { padding: 0 }` is overridden every time it is used**, because both places that use the class set `padding:10px 0` inline in the HTML.
8. **The class `counted` is added by the JavaScript and styled nowhere.** It is harmless, and it does serve as a marker, but nothing reads it.

14.3 Findings about unfinished work

9. **Five "UPDATE" comments remain in `index.html`** telling a future editor to replace placeholder content, even though the commit that populated the page claimed to replace all placeholder content with real agency data. Some were acted on and the comments were not removed.
10. **`ASSETS.md` is stale in two specific ways.** It tells the reader to search `index.html` for `YOUR_CAL_LINK`, which no longer exists there, and to replace the four metric numbers, which appear to have been replaced already.
11. **Not one of the twelve assets `ASSETS.md` calls "required before launch" exists.** The `assets/` folder contains only the checklist itself. In particular there is no favicon and no social preview image, and `index.html` has neither a `<link rel="icon">` nor an `og:image` tag.
12. **The decorative hero tiles display invented engagement numbers** (847K views, 234K impressions, 1.2M, 512K, 1.8M, 19K reactions). They are marked `aria-hidden` and are plainly ornamental, but a visitor reads them as claims. This is worth flagging given that the agency's main site was built specifically to avoid fabricated numbers.
13. **`netlify.toml` configures a host that is not used.** The deploy path is `rsync` to a VPS running Caddy. Netlify appears nowhere else in the repository.

14.4 Findings about accessibility

14. **The counting numbers ignore the reduced-motion preference.** The stylesheet honours `prefers-reduced-motion` for every CSS animation, but the counter is animated in JavaScript over 2.2 seconds and never checks that setting. A visitor who has asked their system to reduce

motion still gets moving numbers. This is the one accessibility defect with a clear fix, given in section 7.3.

15. `#1967ff` **on the dark background is a marginal contrast choice** for the small uppercase "kicker" labels and the 11-pixel service tags. This document does not have a measured contrast figure for those exact pairings, so it is raised as worth checking rather than asserted as a failure.

15 The project's story

Parts 1 to 4 and part 8 are fact, taken from the repository and its history. Parts 5, 6 and 7 are advice and judgement, and are labelled as such where they begin.

15.1 Deliverables

What this project actually produced:

- A complete, self-contained one-page marketing website: 597 lines of HTML, 768 of CSS, 79 of JavaScript, with no dependencies to install.
- A hosting configuration for Netlify (`netlify.toml`) that sets four security response headers.
- A GitHub Actions workflow that publishes the site to the agency VPS over SSH, gated on a `[deploy]` marker, with retries on both the host-key fetch and the file copy, and a post-deploy check that the site answers.
- An asset checklist (`ASSETS.md`) naming the twelve files that would be needed to finish the design.

What it did not produce: a live site. The address ai.digitalise.agency serves something else, as established in section 1.2.

15.2 Timeline and effort

The entire history is nine commits across two calendar days.

Date	Commit	Subject
2026-07-28	`0d77d51`	Launch ai.digitalise.agency, a visual-first marketing site
2026-07-28	`4dffc12`	Update booking link to cal.com/davonex/30min
2026-07-28	`d49e034`	Add deploy workflow
2026-07-28	`2c66a7a`	`[deploy]` Replace all placeholder content with real agency data and switch deploy to VPS rsync
2026-07-29	`f6d810f`	`[deploy]` Trigger deploy after VPS secrets added
2026-07-29	`ed5c154`	`[deploy]` Fix VPS deploy key
2026-07-29	`f698421`	`[deploy]` Fix /root traverse permission for deploy user
2026-07-29	`0c3cad1`	`[deploy]` Remove em-dashes from titles, portfolio copy, and comments
2026-07-29	`da508a7`	Move new design into new-design-reference and disable auto-deploy

Table 8: The complete commit history. Subjects are reproduced as written except that commit `0d77d51` uses a long dash where this table uses a comma; that character is not printable in this document's typeface.

Two days, start to finish, and the second day is entirely deployment: four of the five commits on 2026-07-29 are fixing the publishing pipeline, and the fifth switches it off. Nothing in the

history suggests the design itself was difficult. Everything in the history suggests getting it onto a server was.

This is calendar time, not effort. Git records when commits were made and nothing about hours spent. No document in this repository records effort, so none is claimed here.

15.3 Challenges

Each of these is claimed because the repository carries a scar for it, not because it seems plausible.

1. Getting SSH access to the VPS working (three commits). The scars are `f6d810f` ("Trigger deploy after VPS secrets added"), `ed5c154` ("Fix VPS deploy key") and `f698421` ("Fix /root traverse permission for deploy user"). Read in order they tell a complete story: the workflow was written before the secrets existed, then the key was wrong, then the key was right but the `deploy` user could not traverse into `/root` to reach the target folder. The last of those is a genuinely unobvious Unix permission problem, and it is the reason the deploy target is an unusual path.

2. Fragile host-key fetching on GitHub's runners. The workflow wraps `ssh-keyscan` in a five-attempt retry loop with a five-second wait, and wraps `rsync` in a three-attempt loop with a growing wait. Nobody writes two independent retry loops into a fifty-line script unless both steps have failed on them. The sibling repository `salmanadnan-com`, built the same week from the same skeleton, carries a commit literally titled "Retry, runner IP lottery", which confirms the diagnosis: GitHub's runners come from a wide pool of addresses and some of them were being refused.

3. Em-dashes. Commit `0c3cad1` exists solely to remove them from titles, portfolio copy and comments. This is the workspace-wide house rule showing up as real work; the rule is enforced in this document too.

4. The design was overtaken. The final commit is not a fix, it is a retreat: the page was finished and working, and then something else became the live site, so this one was parked. That is the largest challenge recorded here and it was not technical.

15.4 Outcomes

- **Built and complete.** The page is finished to the standard of a design reference: it renders, it is responsive down to a phone, and it honours the reduced-motion preference in CSS.
- **Not live, and deliberately so.** The design is parked under `new-design-reference/` and the publishing workflow is disabled twice over.
- **Superseded.** ai.digitalise.agency answers with a different page built elsewhere, as verified in section 1.2.
- **Never finished as a product.** None of the twelve assets in the launch checklist were ever added, so the portfolio thumbnails are still colour gradients and the client logos are still coloured circles with a letter in them.
- **Dormant.** No commit since 2026-07-29.

15.5 What to do (judgement)

From here on this section is advice, not a record of project decisions.

- **Read `git log` before you touch anything**, because the only statement of this repository's status lives in a commit message.
- **Serve the folder over HTTP to look at it**, never open the file directly, or the absolute stylesheet path will make it look broken.
- **Keep the `[deploy]` convention** if the workflow is ever restored. It is shared across every repository in this workspace and its consistency is the point.

- **Keep the retry loops.** They were paid for.
- **Treat this as a design source.** Its real remaining value is the stylesheet: a coherent, self-contained dark theme in 768 readable lines with no tooling required.

15.6 What not to do (judgement)

- **Do not re-enable the deploy workflow without changing the `rsync` source path.** It would copy a folder rather than a page and take the live site down. Section 13.1.
- **Do not copy the hero tiles into anything real without replacing their numbers.** They are invented, and this agency's main site was built to avoid exactly that.
- **Do not trust `ASSETS.md`.** Two of its instructions refer to things that are no longer in the code.
- **Do not assume this repository is what visitors see.** It is not, and that is the single easiest mistake to make here.
- **Do not delete the repository because it is not live.** The stylesheet is reusable and the deploy workflow encodes three separate lessons about SSH on GitHub runners.

15.7 If you were starting over (judgement)

This subsection is entirely opinion, argued from what the code shows.

Keep exactly as it is: the no-build-step decision, the token block at the top of the stylesheet, the three-behaviour JavaScript, the reduced-motion block, and the `[deploy]` marker convention. For a page of this size, every one of those choices is right, and a modern framework would have added a hundred times the machinery for no benefit a visitor could perceive.

Do differently: write a README on day one saying what the site is and where it is served from, because the absence of that is what makes this repository confusing. Use a relative stylesheet path so the page can be opened off a disk. Self-host the two fonts, matching the agency's own main site and removing the dependency on Google. Add the favicon and social preview image before the first deploy rather than listing them on a checklist. And delete a placeholder comment in the same commit that removes the placeholder.

Drop: `netlify.toml`, which configures a host this project does not use, and the two unused colour tokens.

Built today, it would look almost identical, which is the strongest thing that can be said for it. The only structural change worth making would be to move the four security headers out of `netlify.toml` and into the Caddy configuration on the VPS, so that they actually take effect.

15.8 Open threads

- **Twelve assets never delivered**, listed in `ASSETS.md`: one hero video, three client logos, six portfolio thumbnails, a favicon and a social preview image.
- **The booking link is a placeholder.** `cal.com/davonex/30min` is the Davonex calendar, not a Digitalise AI one. The same placeholder is recorded as a known outstanding item on the agency's main website, so this is a shared loose end rather than a local mistake.
- **Five "UPDATE" comments** remain in `index.html`, each marking work that may or may not have been done.
- **The relationship to `salmanadnan-com` is undocumented.** The two repositories share an identical file list and an identical structure and were built in the same week. Neither says so.
- **The live site's canonical link** points at salmanadnan.com. Somebody needs to fix that in whichever repository produces the live page.

16 Glossary

Attribute
Extra information inside an opening tag, written as a name and a value, such as `href="#book"`.

Breakpoint
A screen width at which the layout changes. This project has two: 900 pixels and 600 pixels.

Class
A label attached to elements so that CSS and JavaScript can find them again. It has no meaning to the browser by itself.

Commit
One recorded change to a repository, with a message explaining it.

CSS
Cascading Style Sheets: the file that controls appearance.

Custom property
A named value in CSS, defined as `-name: value` and read back as `var(-name)`.

Element
One thing on the page, produced by a tag.

Hex colour
A colour written as six characters after a hash, giving red, green and blue amounts.

HTML
HyperText Markup Language: the file that holds the content and its structure.

IntersectionObserver
A browser facility that watches elements and reports when they scroll into view. This project uses it four times.

JavaScript
The programming language that runs in the browser after the page loads.

Media query
A block of CSS that only applies when a condition about the screen is true, such as being narrower than 900 pixels.

Pseudo-class
A selector suffix describing a temporary state, such as `:hover` for “while the mouse is over it”.

rsync
A command-line program that copies files between computers, transferring only what differs.

Selector
The part of a CSS rule that says which elements it applies to.

SSH
Secure Shell: the standard way to connect to and run commands on another computer securely.

Tag
The marker that creates an element, written between angle brackets.

Viewport
The visible area of the page in the browser window.

VPS
Virtual Private Server: a rented computer the agency controls remotely.

Web server
A program that answers requests for files over the network.

Workflow
A file describing a job GitHub runs automatically when something happens in the repository.

A Appendix A: the file ledger

Every path git tracks in this repository, with its tier, its line count, and the section that explains it. This table is what makes the coverage claim in this document checkable rather than merely asserted: run `git ls-files` in the repository and compare.

`.github/`

Path	Coverage	Lines	Section
`.github/workflows-disabled/deploy.yml.disabled`	explained in full	103	7.6

Table 9: Coverage ledger: `.github/`. 1 file, 103 lines.

`new-design-reference/`

Path	Coverage	Lines	Section
`new-design-reference/index.html`	explained in full	597	7.1
`new-design-reference/css/site.css`	explained in full	768	7.2
`new-design-reference/js/main.js`	explained in full	79	7.3
`new-design-reference/netlify.toml`	explained in full	11	7.4
`new-design-reference/assets/ASSETS.md`	explained in full	33	7.5

Table 10: Coverage ledger: `new-design-reference/`. 5 files, 1,488 lines.

Untracked directories (tier 3)

There are none. Running `git status -porcelain -ignored` in this repository produces no output at all: there are no ignored files, no build output, no dependency folder, and not even a `.gitignore`. Every file present on disk is tracked and every tracked file is explained above.

The arithmetic

`git ls-files` lists **6** paths. All **6** are tier 1, explained in full, line by line, in section 7. **0** files are tier 2 (declared and summarized): there is no generated, vendored, minified or binary content in this repository, checked with a search for generator banners which returned nothing. **0** directories are tier 3 (excluded), because nothing on disk is untracked. Total lines accounted for: **1,591**, which is the sum reported by `git ls-files -z | xargs -0 wc -l`.

$6 + 0 = 6$, and $103 + 1{,}488 = 1{,}591$. The ledger reconciles.